BCR-6605 – what to do now

Nine steps. Where each file lives is in the tables; every change is also written out as FIND / REPLACE in case you would rather edit by hand than paste a whole class.

> **The big one:** `HealthAlarmsPreferencePage.performOk()` never called `hac.performApply()`. JFace does not do it for you. So **"Apply and Close" silently discarded everything** – that is why no deviation was ever saved and the Deviation list was empty. The log confirmed it: the word "deviat" does not appear once in 15,723 lines.

Step 1 – how to open each file

In Eclipse press **Ctrl+Shift+T**, type the class name, press Enter. That opens the exact file. Then **Ctrl+A**, paste mine over it, **Ctrl+S**. The package line at the top of every file I sent tells you whether you are in the right one – check it before pasting. If Ctrl+Shift+T offers two hits, pick the one whose package matches the table.

Step 2 – replace these four classes

☐	Class	Package	What changed
	`HealthAlarmsPreferencePage`	`com.eurocopter.mgs.presentation.configuration.preferences.healthalarms`	`performOk()` now calls `hac.performApply()`
	`HealthAlarmsComposite`	`com.eurocopter.mgs.presentation.configuration.preferences.healthalarms`	`reloadAll()` at the end of `performApply()`; duplicate dead field removed; unused import dropped
	`HealthAlarmsPreferenceStore`	`com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.store`	swallowed `continue;` fixed (NPE); popup name list filled; saved models marked so a second Apply cannot duplicate
	`HealthCommonBusiness`	`com.eurocopter.mgs.business.health.impl`	contextual-parameter loop moved inside its null guard (NPE)

All four were built from your own sources, so everything outside those changes is byte-identical.

Complete source of every changed class is at the end of this document.

Step 3 – confirm these three are already in

☐	Class	Package	What it carries
	`WCModelTreeViewer`	`...preferences.healthalarms.viewer`	`editors[POINTS.ordinal()] = getCellEditor(tree, true);` – without it the Points cell is read-only
	`WCTreeCellModifier`	`...preferences.healthalarms.editor`	the `configurationField` split, `case POINTS:`, `getValue` POINTS case, both component `markAsDirty` calls restored
	`WCTreeLabelProvider`	`...preferences.healthalarms.provider`	`case POINTS:` so the value is painted

Replace any that are not in yet.

Step 4 – the missing message key

File: the message bundle that holds `NbSuccessivePtForAlarmRaising`

The log shows `MissingResourceException: key MaxPointsPerHour` at `WCModelTreeViewer.createColumns` line 66. That is why the header reads "Points." – it is falling back to the raw key.

```
MaxPointsPerHour=Points
```

Step 5 – filtering law from the deviation (already applied in the file supplied)

File: AbstractFilteringRuleAlgorithm **Method:** `getFilteredValues(...)`

REQ1 lists the filtering law as a deviation field. It is stored and displayed, but the analyser still reads the configured one. Same pattern as Points, same class.

FIND

```
            if (indicatorConfiguration.getFilteringLaw() != null && isFilteringLawApplicable(indicatorValueAdapters)) {
               applyFilteringLawAlgorithm(indicatorValueAdapters, filteringLaws, indicatorConfiguration.getFilteringLaw());
            }
```

REPLACE

```
            // BCR-6605 a deviation can replace the configured filtering law
            final String filteringLawName = getFilteringLawName(indicatorConfiguration);
            if (filteringLawName != null && isFilteringLawApplicable(indicatorValueAdapters)) {
               applyFilteringLawAlgorithm(indicatorValueAdapters, filteringLaws, filteringLawName);
            }
```

Add this method beside `getDeviatedPoints`

```
   /**
    * BCR-6605 the filtering law to apply: the deviated one when a deviation set it, otherwise the
    * configured one. Like Points, the law belongs to the indicator, so the first severity carrying
    * a deviated law wins.
    *
    * @param conf
    *           the indicator configuration being filtered
    * @return the law name to apply, or null when there is none
    */
   private String getFilteringLawName(final IndicatorConfiguration conf) {
      if (!deviations.isEmpty() && conf.getIndicator() != null) {
         for (Severity severity : Severity.values()) {
            IndicatorDeviation deviation = deviations.get(conf.getIndicator().getId() + "|" + severity.name());
            if (deviation != null && deviation.getFilteringLaw() != null) {
               return deviation.getFilteringLaw().getName();
            }
         }
      }
      return conf.getFilteringLaw();
   }
```

No new imports. With no deviation the method returns `conf.getFilteringLaw()`, so behaviour is unchanged.

Step 6 – rebuild

Rebuild client and server, redeploy. Do not change `persistence.xml` yet.

Step 7 – test

Use "Apply and Close" for at least the first test – that is the path that was broken.

#	Action	Expect
a	Edit **Level**, add TE, **Apply and Close**	"Deviation created on indicator xxx". **No** applicable-flight popup
b	Reopen the Health page	Row green and not editable
c	Deviation list tab	The row is listed – no longer empty
d	Edit **Points** on a RED row, add TE, Apply	The AMBER row of the same indicator follows. This is by design
e	New deviation, press **Apply twice**	**One** row only, not two
f	Edit **Value** only, Apply	Applicable-flight popup, **no** deviation row
g	Edit **Max time without acquisition** only, Apply	Applicable-flight popup, configuration row created
h	Edit **Level** and **Value** together, Apply	Both popups. The new configuration keeps the **old** Level
i	Column header	Reads **Points**, not "Points."

Step 8 – verify in the database

```
SELECT id, te_reference, max_points_per_hour, filtering_law_id, start_date
FROM public.indicator_deviation
ORDER BY start_date DESC LIMIT 20;
```

One row per deviation. No duplicates from test (e).

Step 9 – last, and only once 1 to 8 pass

File: the server `persistence.xml`

This is why the test data vanished on 13-10 – every redeploy drops and recreates every table.

FIND

```
<property name="eclipselink.ddl-generation" value="drop-and-create-tables"/>
```

REPLACE

```
<property name="eclipselink.ddl-generation" value="none"/>
```

Both deviation columns already exist, so nothing needs auto-creating.

Not a bug – do not "fix" these

Behaviour	Why
With no deviation, both severity rows show the same Points	That value comes from `indicatorConfiguration.getMaxPointsPerHour()`, which is one per indicator. Correct. Once deviated, each row keeps its own value – the cascade has been removed
A deviated row is green and completely read-only	`canModify` returns false for a deviated row. Confirmed as wanted

Why the deviations could not reach the filter before – now solved

`HealthAcquisitionAnalyzer` had no `healthHistoryProvider`: not a field, not a parameter of `analyzeHealthAcquisitions`, not a parameter of `analyzeHealthIndicatorValueAcquisitions`. There was simply nothing to call `getDeviations()` on, which is why the earlier one-line attempt would not compile.

It is now threaded through properly. `IHealthHistoryProvider` is added as the last parameter of both methods and handed to the filtering algorithm:

```
    if (healthHistoryProvider != null && filteringRuleAlgorithm instanceof AbstractFilteringRuleAlgorithm<?> deviationAware) {
       deviationAware.setDeviations(healthHistoryProvider.getDeviations());
    }
```

One line is left for you, and Eclipse will point straight at it. After pasting the class, `analyzeHealthAcquisitions(...)` takes one more argument, so its single caller will show a compile error. Open it and add `healthHistoryProvider` as the last argument. That caller almost certainly has one already – it is the same analysis flow that passes a provider to `LearnedThresholdAnalyzer.updateLearnedThresholds(...)`. If it does not, pass `null`: the guard above makes that safe and behaviour stays as it is today.

Parked

Item	Blocked on
Part F – DACOM extraction	An existing extract script or the class that builds it
Part G – automatic purge	The existing purge job class

Full source – every changed class

Each one is complete. Ctrl+Shift+T, type the class name, Ctrl+A, paste, Ctrl+S.

HealthAlarmsPreferencePage.java

Package: `com.eurocopter.mgs.presentation.configuration.preferences.healthalarms`
Change: performOk() now calls hac.performApply()

```
/**************************************/
/** Copyright Airbus Helicopters 2019 **/
/**************************************/
package com.eurocopter.mgs.presentation.configuration.preferences.healthalarms;

import org.eclipse.swt.SWT;
import org.eclipse.swt.layout.GridData;
import org.eclipse.swt.layout.GridLayout;
import org.eclipse.swt.widgets.Button;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Control;
import org.eclipse.ui.IWorkbench;

import com.eurocopter.mgs.core.presentation.i18n.Messages;
import com.eurocopter.mgs.presentation.configuration.preferences.pages.MgsPreferencePage;
import com.eurocopter.mgs.presentation.configuration.store.MgsPreferenceStoreComposer;

/**
 *
 * Health Alarm Preference Page in the Work Cards.
 * <P>
 *
 */
public class HealthAlarmsPreferencePage extends MgsPreferencePage {

   private HealthAlarmsComposite hac;

   /** Store. */
   private final MgsPreferenceStoreComposer store = new MgsPreferenceStoreComposer();

   /**
    * Default constructor. Builds a HealthAlarmsPreferencePage.
    */
   public HealthAlarmsPreferencePage() {
      super(Messages.CORE.getMnemonicEscapedString("AircraftConfigurationSection_health"), GRID);
   }

   @Override
   protected Control createContents(Composite parent) {
      Composite composite = new Composite(parent, SWT.NONE);
      composite.setLayout(new GridLayout(1, false));
      composite.setLayoutData(new GridData(SWT.FILL, SWT.FILL, true, true, 1, 1));

      composite.setLayout(new GridLayout(1, false));
      composite.setLayoutData(new GridData(SWT.FILL, SWT.FILL, true, true, 1, 1));

      hac = new HealthAlarmsComposite(store, composite);
      setTitle(Messages.CORE.getString(getTitle()));
      return hac;
   }

   @Override
   protected void contributeButtons(Composite parent) {
      ((GridLayout) parent.getLayout()).numColumns++;
      Button manageLaws = new Button(parent, SWT.PUSH);
      manageLaws.setText(Messages.CORE.getString("filteringLawManageButton"));
      manageLaws.addListener(SWT.Selection, e -> hac.openFilteringLawDialog());
      setButtonLayoutData(manageLaws);
   }

   @Override
   public void createControl(Composite parent) {
      super.createControl(parent);
      Button defaults = getDefaultsButton();
      if (defaults != null) {
         defaults.setVisible(false);
      }
   }

   @Override
   public void init(IWorkbench workbench) {
      setPreferenceStore(store);
   }

   @Override
   public void performDefaults() {
      if (hac.isValidData()) {
         hac.performDefaults();
         super.performDefaults();
      }
   }

   @Override
   public boolean performCancel() {
      hac.performCancel();
      return super.performCancel();
   }

   @Override
   public void performApply() {
      if (hac.isValidData()) {
         hac.performApply();
         super.performApply();
      }
   }

   @Override
   public boolean performOk() {
      boolean result = false;
      if (hac.isValidData()) {
         // BCR-6605 JFace does NOT call performApply() from performOk(), so without this
         // "Apply and Close" / OK silently discarded every change, including deviations.
         hac.performApply();
         result = super.performOk();
      }
      return result;
   }

   @Override
   public void checkState() {
      setValid(true);
      super.checkState();
   }
}
```

HealthAlarmsComposite.java

Package: com.eurocopter.mgs.presentation.configuration.preferences.healthalarms
Change: reloadAll() after save; duplicate dead field removed

```
/**************************************/
/** Copyright Airbus Helicopters 2019 **/
/**************************************/
package com.eurocopter.mgs.presentation.configuration.preferences.healthalarms;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.eclipse.jface.dialogs.MessageDialog;
import org.eclipse.jface.viewers.ISelectionChangedListener;
import org.eclipse.swt.SWT;
import org.eclipse.swt.graphics.Point;
import org.eclipse.swt.layout.GridData;
import org.eclipse.swt.layout.GridLayout;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Label;
import org.eclipse.swt.widgets.TabFolder;
import org.eclipse.swt.widgets.TabItem;
import org.eclipse.swt.widgets.Tree;
import org.eclipse.swt.widgets.TreeItem;

import com.eurocopter.mgs.business.health.model.WCThresholdModel;
import com.eurocopter.mgs.core.presentation.i18n.Messages;
import com.eurocopter.mgs.datamodel.aircraft.Aircraft;
import com.eurocopter.mgs.presentation.configuration.preferences.health.AircraftSelectionComposite;
import com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.editor.WCTreeCellModifier;
import com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.provider.WCTreeContentProvider;
import com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.provider.WCTreeLabelProvider;
import com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.store.HealthAlarmsPreferenceStore;
import com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.viewer.WCModelTreeViewer;
import com.eurocopter.mgs.presentation.configuration.store.MgsPreferenceStoreComposer;
import com.eurocopter.mgs.presentation.configuration.store.MgsPreferenceStoreManager;

public class HealthAlarmsComposite extends Composite {

   private static final Logger LOGGER = LogManager.getLogger(HealthAlarmsComposite.class);

   /** Hash Map for the health mechanical component. */
   private final Map<Aircraft, WCMechanicalComponentModel> healthWcModelMap;

   private DeviationTableComposite deviationListComposite;

   private DeviationTableComposite deviationHistoryComposite;

   /** TreeViewer for the thresholds. */
   private WCModelTreeViewer thresholdsTreeViewer;

   /**
    * ComboBox - A/C Type.
    */
   private final AircraftSelectionComposite aircraftCombo;

   /**
    * The Key.
    */
   private Aircraft selectedAircraft = null;

   /**
    * Store.
    */
   private final HealthAlarmsPreferenceStore healthAlarmsPreferenceStore;

   /**
    * Default constructor. Builds a HealthAlarmsComposite.
    *
    * @param parentStore
    *           the parent store
    * @param parent
    *           the parent composite
    */
   public HealthAlarmsComposite(MgsPreferenceStoreComposer parentStore, Composite parent) {
      super(parent, SWT.NONE);
      GridLayout layout = new GridLayout();
      layout.marginHeight = 0;
      layout.marginWidth = 0;
      layout.verticalSpacing = 0;
      setLayout(layout);
      setLayoutData(new GridData(SWT.FILL, SWT.FILL, true, true, 1, 1));

      // add the store
      healthAlarmsPreferenceStore = new HealthAlarmsPreferenceStore(this);
      parentStore.addStore(healthAlarmsPreferenceStore);

      // instantiate the hash map
      healthWcModelMap = new HashMap<>();

      // A/C type composite
      aircraftCombo = new AircraftSelectionComposite(this);
      // BCR-6605 Message display
      aircraftCombo.setLayoutData(new GridData(SWT.FILL, SWT.TOP, true, false));
      aircraftCombo.forceFocus();

      // set a space between composite
      new Label(this, SWT.NONE);

      TabFolder tabFolder = new TabFolder(this, SWT.NONE);
      tabFolder.setLayoutData(new GridData(SWT.FILL, SWT.FILL, true, true));

      // Tab 1 - Configuration: holds the existing tree
      TabItem configurationTab = new TabItem(tabFolder, SWT.NONE);
      configurationTab.setText(Messages.CORE.getString("deviationTabConfiguration"));
      Composite configurationComposite = new Composite(tabFolder, SWT.NONE);
      GridLayout tabLayout = new GridLayout();
      tabLayout.marginHeight = 0;
      tabLayout.marginWidth = 0;
      configurationComposite.setLayout(tabLayout);
      configurationTab.setControl(configurationComposite);
      createTreeViewer(configurationComposite);

      // Tab 2 - Deviation list
      TabItem deviationListTab = new TabItem(tabFolder, SWT.NONE);
      deviationListTab.setText(Messages.CORE.getString("deviationTabList"));
      deviationListComposite = new DeviationTableComposite(tabFolder, healthAlarmsPreferenceStore, false, this::reloadAll);
      deviationListTab.setControl(deviationListComposite);

      // Tab 3 - Deviation history
      TabItem deviationHistoryTab = new TabItem(tabFolder, SWT.NONE);
      deviationHistoryTab.setText(Messages.CORE.getString("deviationTabHistory"));
      deviationHistoryComposite = new DeviationTableComposite(tabFolder, healthAlarmsPreferenceStore, true, this::reloadAll);
      deviationHistoryTab.setControl(deviationHistoryComposite);

      Composite lawButtonBar = new Composite(this, SWT.NONE);
      lawButtonBar.setLayout(new GridLayout(1, false));
      lawButtonBar.setLayoutData(new GridData(SWT.END, SWT.CENTER, true, false));

      aircraftCombo.getComboViewer().addSelectionChangedListener((ISelectionChangedListener) event -> {
         selectedAircraft = aircraftCombo.getSelectedAircraft();
         if (selectedAircraft != null) {
            healthAlarmsPreferenceStore.initMechanicalComponentData(selectedAircraft);
            WCMechanicalComponentModel aircraftComponentModel = healthAlarmsPreferenceStore.getHealthModels(selectedAircraft);
```

```
            refreshViewer(selectedAircraft, aircraftComponentModel);
            deviationListComposite.setAircraft(selectedAircraft);
            deviationHistoryComposite.setAircraft(selectedAircraft);
         }
      });
   }

   /**
    * Create the thresholds Tree Viewer.
    *
    * @param composite
    *           Parent Composite
    */
   private void createTreeViewer(Composite composite) {
      final Tree tree = new Tree(composite, SWT.BORDER | SWT.H_SCROLL | SWT.V_SCROLL | SWT.FULL_SELECTION);
      tree.setLayoutData(new GridData(SWT.FILL, SWT.FILL, true, true, 2, 1));
      tree.setHeaderVisible(true);
      tree.setLinesVisible(true);
      thresholdsTreeViewer = new WCModelTreeViewer(tree, healthWcModelMap, healthAlarmsPreferenceStore);
      thresholdsTreeViewer.setLabelProvider(new WCTreeLabelProvider());
      thresholdsTreeViewer.setContentProvider(new WCTreeContentProvider());
      thresholdsTreeViewer.setCellModifier(new WCTreeCellModifier(thresholdsTreeViewer));

      tree.addListener(SWT.MouseDown, event -> {
         TreeItem item = tree.getItem(new Point(event.x, event.y));
         if (item != null && item.getData() instanceof WCThresholdModel model && model.isDeviated()) {
            MessageDialog.openInformation(tree.getShell(), Messages.CORE.getString("deviationAlreadySetTitle"), Messages.CORE.getString("deviationAlreadySet"));
         }
      });
   }

   /**
    * Refresh Viewer.
    *
    * @param wcModel
    *           List<HealthWCModel> models.
    */
   private void refreshViewer(Aircraft aircraft, WCMechanicalComponentModel wcModel) {
      thresholdsTreeViewer.setData("aircraft", aircraft);
      thresholdsTreeViewer.setSelectedAircraft(aircraft);
      thresholdsTreeViewer.setInput(wcModel);
      thresholdsTreeViewer.expandAll();
      thresholdsTreeViewer.refresh();
   }

   /**
    * Perform Apply for the preference page.
    */

   /*
    * public void performApply() { healthAlarmsPreferenceStore.innerSave(); try { MgsPreferenceStoreManager.saveAircraftConfigurations();
    *
    * healthAlarmsPreferenceStore.saveDeviations(); MgsPreferenceStoreManager.clearAircraftConfigurations(); reloadAll();
    *
    * MgsPreferenceStoreManager.clearAircraftConfigurations(); performDefaults();
    *
    * } catch (Exception e) { LOGGER.error("Error while saving health preferences", e); } }
    */

   public void performApply() {
      healthAlarmsPreferenceStore.innerSave();
      try {
         MgsPreferenceStoreManager.saveAircraftConfigurations();
         /*
          * if (MgsPreferenceStoreManager.wasLastSaveConfirmed()) { healthAlarmsPreferenceStore.saveDeviations(); }
          */
         healthAlarmsPreferenceStore.saveDeviations();
         List<String> deviated = healthAlarmsPreferenceStore.getLastSavedDeviationIndicatorNames();
         if (!deviated.isEmpty()) {
            MessageDialog.openInformation(getShell(),
                  Messages.CORE.getString("deviation.created.title"),
                  Messages.CORE.getString("deviation.created.message") + String.join(", ", deviated));
         }

         MgsPreferenceStoreManager.clearAircraftConfigurations();
         // BCR-6605 refresh the tree and both deviation tabs so the saved rows turn green
         reloadAll();
      } catch (Exception e) {
         LOGGER.error("Error while saving health preferences", e);
      }
   }

   /**
    * Validate data before setting them.
    *
    * @return true if data are valid, false otherwise
    */
   public boolean isValidData() {
      return healthAlarmsPreferenceStore.isValidData(healthWcModelMap);
   }

   /**
    * Perform Defaults for the preference page.
    */
   public void performDefaults() {
      if (selectedAircraft != null) {
         healthWcModelMap.clear();
         refreshViewer(selectedAircraft, healthAlarmsPreferenceStore.loadHealthModels(selectedAircraft));
      }
   }

   /**
    * Cancel was pressed.
    */
   public void performCancel() {
      // empty the store, otherwise data will be saved because of our handling of needSaving method.
      healthAlarmsPreferenceStore.emptyData();
   }

   /**
    * BCR-6605 reload the tree and both deviation tabs.
    */
   private void reloadAll() {
      if (selectedAircraft == null) {
         return;
      }
      healthWcModelMap.clear();
      refreshViewer(selectedAircraft, healthAlarmsPreferenceStore.loadHealthModels(selectedAircraft));
      deviationListComposite.setAircraft(selectedAircraft);
      deviationHistoryComposite.setAircraft(selectedAircraft);
   }

   public void openFilteringLawDialog() {
      if (selectedAircraft != null) {
         new FilteringLawDialog(healthAlarmsPreferenceStore, selectedAircraft).open();
         reloadAll();
      }
   }

}
```

HealthAlarmsPreferenceStore.java

Package: com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.store
Change: swallowed continue fixed; popup list filled; models marked saved

```
/****************************************/
/** Copyright Airbus Helicopters 2019 **/
/****************************************/
package com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.store;

import java.text.MessageFormat;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.Date;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import java.util.Objects;
import java.util.Set;
import java.util.TreeMap;
import java.util.TreeSet;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.eclipse.jface.dialogs.MessageDialog;
import org.eclipse.jface.util.SafeRunnable;
import org.eclipse.ui.statushandlers.StatusManager;
import org.osgi.framework.BundleContext;
import org.osgi.framework.FrameworkUtil;

import com.eurocopter.mgs.business.health.api.IHealthCommonBusiness;
import com.eurocopter.mgs.business.health.model.IndicatorModel;
import com.eurocopter.mgs.business.health.model.WCThresholdBaseline;
import com.eurocopter.mgs.business.health.model.WCThresholdModel;
import com.eurocopter.mgs.core.presentation.i18n.Messages;
import com.eurocopter.mgs.datamodel.aircraft.Aircraft;
import com.eurocopter.mgs.datamodel.alarm.Severity;
import com.eurocopter.mgs.datamodel.health.monitoring.HealthMechanicalComponent;
import com.eurocopter.mgs.datamodel.health.monitoring.HealthMechanicalComponentConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.HealthMonitoringConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.Indicator;
import com.eurocopter.mgs.datamodel.health.monitoring.IndicatorConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.ThresholdLevel;
import com.eurocopter.mgs.datamodel.health.monitoring.ThresholdType;
import com.eurocopter.mgs.health.activator.Activator;
import com.eurocopter.mgs.presentation.common.MgsSession;
import com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.HealthAlarmsComposite;
import com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.WCHealthMechanicalComponentModel;
import com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.WCMechanicalComponentModel;
import com.eurocopter.mgs.presentation.configuration.store.MgsPreferenceStore;
import com.eurocopter.mgs.presentation.configuration.store.MgsPreferenceStoreManager;
import com.eurocopter.mgs.presentation.utils.AircraftExceptionHandler;
import com.eurocopter.mgs.service.aircraft.dto.AircraftConfigurationHistoryDto;
import com.eurocopter.mgs.service.aircraft.exception.AircraftConcurrentAccessException;
import com.eurocopter.mgs.service.exception.EntityNotFoundException;
import com.eurocopter.mgs.service.health.common.dto.FilteringLawDto;
import com.eurocopter.mgs.service.health.common.dto.IndicatorConfigurationDto;
import com.eurocopter.mgs.service.health.common.dto.IndicatorDeviationDto;
import com.eurocopter.mgs.service.health.common.dto.MechanicalComponentConfigurationDto;
import com.eurocopter.mgs.service.health.common.dto.ThresholdConfigurationDto;
import com.google.common.collect.ArrayListMultimap;
import com.google.common.collect.Multimap;

/**
 * Preference store for health alarms.
 */
public class HealthAlarmsPreferenceStore extends MgsPreferenceStore {

   private static final Logger LOGGER = LogManager.getLogger(HealthAlarmsPreferenceStore.class);
   /** data. */
   private Map<Aircraft, WCMechanicalComponentModel> data = new HashMap<>();

   /**
    * List of MCModels
    */
   private WCMechanicalComponentModel rootMechanicalComponents;

   // field, beside lawNameToId
   private final Map<String, String> indicatorDisplayNames = new HashMap<>();

   private final Map<String, String> lawNameToId = new HashMap<>();

   /** Composite parent. */
   private final HealthAlarmsComposite parent;

   private final BundleContext healthBundleContext = FrameworkUtil.getBundle(IHealthCommonBusiness.class)
         .getBundleContext();
   private final IHealthCommonBusiness healthBusiness = healthBundleContext
         .getService(healthBundleContext.getServiceReference(IHealthCommonBusiness.class));

   private final List<String> lastSavedDeviationIndicatorNames = new ArrayList<>();

   private static final Comparator<Severity> SEVERITY_COMPARATOR = (severity1, severity2) -> {
      if (severity1 == severity2) {
         return 0;
      }
      if (severity1 == Severity.RED) {
         return -1;
      }
      return 1;
   };

   /**
    * Constructor with argument. Builds a HealthAlarmsPreferenceStore.
    */
   public HealthAlarmsPreferenceStore(HealthAlarmsComposite parent) {
      this.parent = parent;
   }

   /**
    * Initialize the original mechanical component data.
    *
    * @param aircraft
    *           the aircraft model
    */
   public void initMechanicalComponentData(Aircraft aircraft) {
      retrieveAircraftMechanicalComponents(aircraft);
   }

   @Override
   public boolean needsSaving() {
      return !data.isEmpty();
   }

   @Override
   public void innerLoad() {
   }

   @Override
   public void innerSave() {

      for (Entry<Aircraft, WCMechanicalComponentModel> modelEntry : data.entrySet()) {
         Aircraft aircraft = modelEntry.getKey();
         HealthMonitoringConfiguration configuration;
         AircraftConfigurationHistoryDto aircraftConfigurationHistoryDto;
         try {
```

```
            configuration = getConfiguration(aircraft);
            aircraftConfigurationHistoryDto = MgsPreferenceStoreManager.getAircraftConfigurationDto(aircraft);
         } catch (EntityNotFoundException e) {
            LOGGER.warn("Entity not found exception", e);
            AircraftConcurrentAccessException wrapper = new AircraftConcurrentAccessException(aircraft, true);
            StatusManager.getManager().handle(AircraftExceptionHandler.getStatusForAircraftException(wrapper, "update", Activator.PLUGIN_ID), StatusManager.SHOW);
            continue;
         }
         // Retrieve the root node element
         WCMechanicalComponentModel wcModel = modelEntry.getValue();

         // Create maps to associate JPA entities with WC* instances
         Map<HealthMechanicalComponent, WCHealthMechanicalComponentModel> wcMechanicalComponentModels = new HashMap<>();
         Map<Indicator, Set<WCThresholdModel>> thresholdModels = new HashMap<>();
         getModels(wcModel, wcMechanicalComponentModels, thresholdModels);

         for (Map.Entry<HealthMechanicalComponent, HealthMechanicalComponentConfiguration> mcEntry : configuration.getMechanicalComponentConfigurations().entrySet()) {
            HealthMechanicalComponent mechanicalComponent = mcEntry.getKey();
            MechanicalComponentConfigurationDto mechanicalComponentConfigurationDto = new MechanicalComponentConfigurationDto(mechanicalComponent.getId());
            WCHealthMechanicalComponentModel mechanicalComponentModel = wcMechanicalComponentModels.get(mechanicalComponent);
            mechanicalComponentConfigurationDto.setWorkcardReference(mechanicalComponentModel.getWorkcardReference());
            mechanicalComponentConfigurationDto.setMaxTimeWOAcquisition(mechanicalComponentModel.getMaxtimeWithoutAcquitision());

            for (Map.Entry<Indicator, IndicatorConfiguration> indicatorEntry : mcEntry.getValue().getIndicatorConfigurations().entrySet()) {
               Indicator indicator = indicatorEntry.getKey();
               if (!thresholdModels.containsKey(indicator)) {
                  // This indicator is likely inactive and has no current configuration to copy
                  continue;
               }
               Set<WCThresholdModel> indicatorThresholdModels = thresholdModels.get(indicator);
               IndicatorConfigurationDto indicatorConfigurationDto = new IndicatorConfigurationDto(indicator.getId());
               for (WCThresholdModel wcThresholdModel : indicatorThresholdModels) {
                  if (wcThresholdModel.isEnabled()) {
                     // BCR-6605 deviation fields from the baseline, configuration fields as edited
                     final WCThresholdBaseline baseline = wcThresholdModel.getBaseline() != null
                           ? wcThresholdModel.getBaseline()
                           : new WCThresholdBaseline(wcThresholdModel, wcThresholdModel.getFilteringLawName());
                     final ThresholdConfigurationDto thresholdConfigurationDto;
                     if (baseline.getType() == ThresholdType.FIXED) {
                        thresholdConfigurationDto = new ThresholdConfigurationDto(
                              wcThresholdModel.getSeverity(), baseline.getLevel(), baseline.getType(),
                              wcThresholdModel.getDefaultValue(),
                              wcThresholdModel.getNbSuccessivePoints(),
                              wcThresholdModel.getMinimumEngineOilTemperature());
                     } else {
                        thresholdConfigurationDto = new ThresholdConfigurationDto(
                              wcThresholdModel.getSeverity(), baseline.getLevel(), baseline.getType(),
                              wcThresholdModel.getLearningTime(),
                              baseline.getMinValue(), baseline.getMaxValue(),
                              wcThresholdModel.getDefaultValue(), wcThresholdModel.getMinimumDeviation(),
                              wcThresholdModel.getMeanFactor(), wcThresholdModel.getDeviationCoefficient(),
                              wcThresholdModel.getNbSuccessivePoints(),
                              wcThresholdModel.getMinimumEngineOilTemperature());
                     }
                     thresholdConfigurationDto.setWorkcard(baseline.getWorkCardReference());
                     indicatorConfigurationDto.getThresholdConfigurationDtos().add(thresholdConfigurationDto);
                  }
               }

               mechanicalComponentConfigurationDto.getIndicatorConfigurationDtos().add(indicatorConfigurationDto);
            }
            aircraftConfigurationHistoryDto.getMechanicalComponentConfigurations()
                  .add(mechanicalComponentConfigurationDto);
         }

         aircraftConfigurationHistoryDto.setIndicatorConfigurationUpdated(true);
      }
   }

   private void getModels(WCMechanicalComponentModel wcModel,
         Map<HealthMechanicalComponent, WCHealthMechanicalComponentModel> mechanicalComponents,
         Map<Indicator, Set<WCThresholdModel>> thresholdModels) {
      for (WCMechanicalComponentModel subModel : wcModel.getSubHealthWCModel()) {
         getModels(subModel, mechanicalComponents, thresholdModels);
      }
      for (WCHealthMechanicalComponentModel healthMechanicalComponentModel : wcModel.getHealthComponentModels()) {
         mechanicalComponents.put(healthMechanicalComponentModel.getHealthMechanicalComponent(),
               healthMechanicalComponentModel);
         for (WCThresholdModel wcThresholdModel : healthMechanicalComponentModel.getThresholdModels()) {
            if (!thresholdModels.containsKey(wcThresholdModel.getIndicator())) {
               thresholdModels.put(wcThresholdModel.getIndicator(), new HashSet<>());
            }
            thresholdModels.get(wcThresholdModel.getIndicator()).add(wcThresholdModel);
         }
      }
   }

   private HealthMonitoringConfiguration getConfiguration(Aircraft aircraft) throws EntityNotFoundException {
      try {
         return MgsPreferenceStoreManager.getConfiguration(aircraft, HealthMonitoringConfiguration.TYPE,
               HealthMonitoringConfiguration.class);
      } catch (EntityNotFoundException e) {
         LOGGER.error("Could not load health configuration for aircraft. Register number - Serial number = {}",
               aircraft.getRegisterNumber(), e);
         throw e;
      }
   }

   /**
    * Load data.
    *
    * @param aircraft
    *           the specified aircraft
    * @return a list of health data
    */
   public WCMechanicalComponentModel loadHealthModels(Aircraft aircraft) {
      emptyData();
      retrieveAircraftMechanicalComponents(aircraft);
      // save it
      data.put(aircraft, rootMechanicalComponents);
      rootMechanicalComponents.setName(Messages.CORE.getString("Label.HealthComponents"));
      WCMechanicalComponentModel all = new WCMechanicalComponentModel();
      all.getSubHealthWCModel().add(rootMechanicalComponents);
      all.setName(Messages.CORE.getString("Label.HealthComponents"));
      return all;
   }

   /**
    * Retrieve the data related to an Aircraft.
    *
    * @param aircraft
    *           the aircraft
    * @return the list of health data
    */
   public WCMechanicalComponentModel getHealthModels(Aircraft aircraft) {
      final WCMechanicalComponentModel result;
      if (data.containsKey(aircraft)) {
         result = data.get(aircraft);
      } else {
         // load it
         result = loadHealthModels(aircraft);
      }

      if (!Messages.CORE.getString("Label.HealthComponents").equals(result.getName())) {
         WCMechanicalComponentModel all = new WCMechanicalComponentModel();
         all.getSubHealthWCModel().add(result);
         all.setName(Messages.CORE.getString("Label.HealthComponents"));
         return all;
      }
      return result;

   }
```

```
   /**
    * Empty the data of the store.
    */
   public void emptyData() {
      data.clear();
   }

   /**
    * Indicates whether data are valid or not.
    *
    * @param dataToBeValidated
    *           to be validated
    * @return true if data are valid, false otherwise
    */
   public boolean isValidData(Map<Aircraft, WCMechanicalComponentModel> dataToBeValidated) {

      // for each aircraft
      for (Entry<Aircraft, WCMechanicalComponentModel> entry : dataToBeValidated.entrySet()) {

         // retrieve the whole set of thresholds of the current aircraft
         Multimap<Indicator, WCThresholdModel> wcThresholdMap = getComponentThresholds(entry.getValue());
         for (Indicator indicator : wcThresholdMap.keySet()) {
            WCThresholdModel redModel = null;
            WCThresholdModel amberModel = null;
            for (WCThresholdModel currentModel : wcThresholdMap.get(indicator)) {
               switch (currentModel.getType()) {
                  case FIXED:
                     if (currentModel.getDefaultValue() == null) {
                        // fixed thresholds without value
                        MessageDialog.openError(parent.getShell(),
                              Messages.CORE.getString("healthAlarmPageDataInvalidTitle"),
                              MessageFormat.format(
                                    Messages.CORE.getMnemonicEscapedString("healthAlarmPageDataInvalidMessage"),
                                    entry.getKey().getRegisterNumber(),
                                    currentModel.getHealthMechanicalComponentName() + "/"
                                          + currentModel.getIndicator()
                                          + " (fixed threshold value cannot be undefined)"));
                        return false;
                     }
                     break;

                  case LEARNED:
                     if (currentModel.getMinValue() != null) {
                        final double min;
                        final double max;
                        if (currentModel.getLevel() == ThresholdLevel.HIGH) {
                           min = currentModel.getMinValue();
                           max = currentModel.getMaxValue();
                        } else {
                           min = currentModel.getMaxValue();
                           max = currentModel.getMinValue();
                        }
                        if (min >= max) {
                           MessageDialog.openError(parent.getShell(),
                                 Messages.CORE.getString("healthAlarmPageDataInvalidTitle"),
                                 MessageFormat.format(
                                       Messages.CORE
                                             .getMnemonicEscapedString("healthAlarmPageDataInvalidMessage"),
                                       entry.getKey().getRegisterNumber(),
                                       currentModel.getHealthMechanicalComponentName() + "/"
                                             + currentModel.getIndicator() + " (check min and max values)"));
                           return false;
                        }
                     }
                     break;
                  default:
                     throw new IllegalStateException("Unhandled threshold type : " + currentModel.getType());
               }
               if (currentModel.getSeverity() == Severity.RED) {
                  redModel = currentModel;
               } else {
                  amberModel = currentModel;
               }
            }

            if (redModel != null && amberModel != null) {

               boolean valid = true;
               if (amberModel.getDefaultValue() == null || redModel.getDefaultValue() == null) {
                  continue;
               } else if (amberModel.getLevel() == redModel.getLevel()) {
                  boolean highAmberHigherThanHighRed = amberModel.getLevel() == ThresholdLevel.HIGH
                        && amberModel.getDefaultValue() > redModel.getDefaultValue();
                  // threshold
                  boolean lowAmberSmallerThanLowRed = amberModel.getLevel() == ThresholdLevel.LOW
                        && amberModel.getDefaultValue() < redModel.getDefaultValue();
                  if (highAmberHigherThanHighRed || lowAmberSmallerThanLowRed) {
                     valid = false;
                  }
               } else if (amberModel.getLevel() == ThresholdLevel.HIGH
                     && amberModel.getDefaultValue() < redModel.getDefaultValue()) {
                  // threshold
                  valid = false;
               } else if ((redModel.getLevel() == ThresholdLevel.HIGH)
                     && (redModel.getDefaultValue() < amberModel.getDefaultValue())) {
                  // threshold
                  valid = false;
               }
               if (!valid) {
                  MessageDialog.openError(parent.getShell(),
                        Messages.CORE.getString("healthAlarmPageDataInvalidTitle"),
                        MessageFormat.format(
                              Messages.CORE.getMnemonicEscapedString("healthAlarmPageDataInvalidMessage"),
                              entry.getKey().getRegisterNumber(),
                              amberModel.getHealthMechanicalComponentName() + ">"
                                    + amberModel.getIndicator().getName()));
                  return false;
               }
            }
         }
      }

      List<WCThresholdModel> deviatedModels = new ArrayList<>();
      for (Entry<Aircraft, WCMechanicalComponentModel> entry : dataToBeValidated.entrySet()) {
         for (WCThresholdModel model : getComponentThresholds(entry.getValue()).values()) {
            if (model.isDeviated() || model.isModifiedAgainstBaseline()) {
               deviatedModels.add(model);
            }
         }
      }
      for (WCThresholdModel model : deviatedModels) {
         if (model.getTeReference() == null || model.getTeReference().trim().isEmpty()) {
            MessageDialog.openError(parent.getShell(),
                  Messages.CORE.getString("healthAlarmPageDataInvalidTitle"), MessageFormat.format(Messages.CORE.getString("deviationErrorTeMandatory"),
model.getIndicator().getName()));
            return false;
         }
      }
      for (WCThresholdModel model : deviatedModels) {
         model.setDeviated(true);
      }
      this.data = dataToBeValidated;
      return true;
   }

   /**
    * Retrieve the list of the whole threshold models for a specific component.
    *
    * @param component
    *           the component from which thresholds models have to be retrieved
    * @return a list of thresholds models
```

```
 */
private Multimap<Indicator, WCThresholdModel> getComponentThresholds(WCMechanicalComponentModel component) {
   Multimap<Indicator, WCThresholdModel> thresholdModels = ArrayListMultimap.create();

   // add the list of thresholds models to the result, if they exist
   for (WCHealthMechanicalComponentModel healthMechanicalComponentModel : component.getHealthComponentModels()) {
      for (WCThresholdModel thresholdModel : healthMechanicalComponentModel.getThresholdModels()) {
         if (thresholdModel.isEnabled()) {
            thresholdModels.put(thresholdModel.getIndicator(), thresholdModel);
         }
      }
   }
   for (WCMechanicalComponentModel mechanicalComponentModel : component.getSubHealthWCModel()) {
      thresholdModels.putAll(getComponentThresholds(mechanicalComponentModel));
   }
   return thresholdModels;
}

/**
 * Retrieves the aircraft monitored mechanical components and indicators with their status from the data base.
 */
private void retrieveAircraftMechanicalComponents(final Aircraft aircraft) {
   SafeRunnable.run(new SafeRunnable() {
      @Override
      public void run() {
         try {
            rootMechanicalComponents = createAllMechanicalComponents(aircraft).get("");
         } catch (EntityNotFoundException e) {
            LOGGER.error("Cannot find mechanical components for aircaft : {}", aircraft, e);
         }
      }
   });
}

/**
 * Build Health node Tree hierarchy
 */
private Map<String, WCMechanicalComponentModel> createAllMechanicalComponents(final Aircraft aircraft)
      throws EntityNotFoundException {
   Map<String, WCMechanicalComponentModel> createdObjects = new HashMap<>();
   Map<Indicator, IndicatorModel> indicatorModels;
   try {
      indicatorModels = healthBusiness.getIndicatorModels(MgsSession.getInstance().getUser().getId(),
            aircraft.getId());
   } catch (EntityNotFoundException e1) {
      LOGGER.warn("Unable to retrieve indicator models", e1);
      return createdObjects;
   }

   Map<HealthMechanicalComponent, HealthMechanicalComponentConfiguration> hmcConfigurations = healthBusiness.getActiveComponentsAndConfiguration(aircraft.getId());
   Map<String, IndicatorDeviationDto> deviationMap = new HashMap<>();
   try {
      for (IndicatorDeviationDto dto : healthBusiness.findActiveDeviations(aircraft.getId())) {
         deviationMap.put(dto.getIndicatorId() + "|" + dto.getSeverity(), dto);
      }
   } catch (Exception e) {
      LOGGER.warn("Could not load active deviations", e);
   }

   for (Entry<HealthMechanicalComponent, HealthMechanicalComponentConfiguration> entry : hmcConfigurations
         .entrySet()) {
      HealthMechanicalComponent healthMechanicalComponent = entry.getKey();
      int position = healthMechanicalComponent.getPath()
            .lastIndexOf(HealthMechanicalComponent.HMC_PATH_SEPARATOR);
      String parentPath = healthMechanicalComponent.getPath().substring(0, position);
      WCMechanicalComponentModel parentComponent = createdObjects.get(parentPath);
      if (parentComponent == null) {
         parentComponent = createMechanicalComponent(parentPath, createdObjects);
      }
      // List<WCThresholdModel> thresholdModels = getThresholdModels(healthMechanicalComponent, indicatorModels);
      List<WCThresholdModel> thresholdModels = getThresholdModels(healthMechanicalComponent,
            indicatorModels, entry.getValue().getIndicatorConfigurations(), deviationMap);
      WCHealthMechanicalComponentModel wcModel = new WCHealthMechanicalComponentModel(healthMechanicalComponent,
            thresholdModels, entry.getValue().getWorkcard(),
            entry.getValue().getMaximumTimeWithoutAcquistion());
      parentComponent.getHealthComponentModels().add(wcModel);
   }
   return createdObjects;
}

private WCMechanicalComponentModel createMechanicalComponent(String path,
      Map<String, WCMechanicalComponentModel> createdObjects) {
   int position = path.lastIndexOf(HealthMechanicalComponent.HMC_PATH_SEPARATOR);
   WCMechanicalComponentModel newMCModel = new WCMechanicalComponentModel();
   newMCModel.setName(path.substring(position + 1));
   createdObjects.put(path, newMCModel);
   if (position >= 0) {
      String parentPath = path.substring(0, position);
      WCMechanicalComponentModel parentComponent = createdObjects.get(parentPath);
      if (parentComponent == null) {
         parentComponent = createMechanicalComponent(parentPath, createdObjects);
      }
      parentComponent.getSubHealthWCModel().add(newMCModel);
   }
   return newMCModel;
}

private List<WCThresholdModel> getThresholdModels(HealthMechanicalComponent healthMechanicalComponent,
      Map<Indicator, IndicatorModel> indicatorModels, Map<Indicator, IndicatorConfiguration> indicatorConfigurations,
      Map<String, IndicatorDeviationDto> deviationMap) {
   Severity[] severities = { Severity.RED, Severity.AMBER };
   List<WCThresholdModel> hmcThresholdModels = new ArrayList<>();
   boolean odd = true;

   Set<Indicator> sortedSet = new TreeSet<>((i1, i2) -> i1.getName().compareTo(i2.getName()));
   sortedSet.addAll(healthMechanicalComponent.getIndicators().values());
   for (Indicator indicator : sortedSet) {
      Map<Severity, WCThresholdModel> indicatorThresholdsMap = new TreeMap<>(SEVERITY_COMPARATOR);
      IndicatorModel indicatorModel = indicatorModels.get(indicator);
      String path = healthMechanicalComponent.getPath().replace(String.valueOf(HealthMechanicalComponent.HMC_PATH_SEPARATOR), " - ");
      if (path.startsWith(" - ")) {
         path = path.substring(3);
      }
      indicatorDisplayNames.put(indicator.getId(), path + " - " + indicator.getName());
      for (WCThresholdModel thresholdModel : indicatorModel.getThresholdModels()) {
         thresholdModel.setOdd(odd);
         indicatorThresholdsMap.put(thresholdModel.getSeverity(), thresholdModel);
      }
      for (Severity severity : severities) {
         if (!indicatorThresholdsMap.containsKey(severity)) {
            WCThresholdModel thresholdModel = new WCThresholdModel(indicator,
                  healthMechanicalComponent.getFunctionalId());
            thresholdModel.setSeverity(severity);
            thresholdModel.setOdd(odd);
            indicatorThresholdsMap.put(severity, thresholdModel);
         }
      }
      // stored deviation
      IndicatorConfiguration indicatorConfiguration = indicatorConfigurations.get(indicator);
      String law = indicatorConfiguration != null ? indicatorConfiguration.getFilteringLaw() : null;
      String icId = indicatorConfiguration != null ? indicatorConfiguration.getId() : null;
      Integer points = indicatorConfiguration != null ? indicatorConfiguration.getMaxPointsPerHour() : null;
      for (WCThresholdModel thresholdModel : indicatorThresholdsMap.values()) {
         thresholdModel.setFilteringLawName(law);
         thresholdModel.setIndicatorConfigurationId(icId);
         thresholdModel.setMaxPointsPerHour(points);
         thresholdModel.setBaseline(new WCThresholdBaseline(thresholdModel, law));
         IndicatorDeviationDto deviation = deviationMap.get(indicator.getId() + "|" + thresholdModel.getSeverity().name());
         if (deviation != null) {
            thresholdModel.setDeviated(true);
            thresholdModel.setDeviationId(deviation.getId());
            thresholdModel.setTeReference(deviation.getTeReference());
```

```
               thresholdModel.setDeviationStartDate(deviation.getStartDate());
               thresholdModel.setDeviationEndDate(deviation.getEndDate());
               if (deviation.getThresholdLevel() != null) {
                  thresholdModel.setLevel(ThresholdLevel.valueOf(deviation.getThresholdLevel()));
               }
               if (deviation.getThresholdType() != null) {
                  thresholdModel.setType(ThresholdType.valueOf(deviation.getThresholdType()));
               }
               if (deviation.getThresholdValue() != null) {
                  thresholdModel.setDefaultValue(deviation.getThresholdValue());
               }
               if (deviation.getMinValue() != null) {
                  thresholdModel.setMinValue(deviation.getMinValue());
               }
               if (deviation.getMaxValue() != null) {
                  thresholdModel.setMaxValue(deviation.getMaxValue());
               }
               if (deviation.getLearningTime() != null) {
                  thresholdModel.setLearningTime(deviation.getLearningTime());
               }
               if (deviation.getWorkcard() != null) {
                  thresholdModel.setWorkCardReference(deviation.getWorkcard());
               }
               if (deviation.getMinimumDeviation() != null) {
                  thresholdModel.setMinimumDeviation(deviation.getMinimumDeviation());
               }
               if (deviation.getMeanFactor() != null) {
                  thresholdModel.setMeanFactor(deviation.getMeanFactor());
               }
               if (deviation.getDeviationCoefficient() != null) {
                  thresholdModel.setDeviationCoefficient(deviation.getDeviationCoefficient());
               }
               if (deviation.getNbSuccessivePoints() != null) {
                  thresholdModel.setNbSuccessivePoints(deviation.getNbSuccessivePoints());
               }
               if (deviation.getMaxPointsPerHour() != null) {
                  thresholdModel.setMaxPointsPerHour(deviation.getMaxPointsPerHour());
               }
               if (deviation.getFilteringLawName() != null) {
                  thresholdModel.setFilteringLawName(deviation.getFilteringLawName());
                  thresholdModel.setFilteringLawId(deviation.getFilteringLawId());
               }
            }

         }
         hmcThresholdModels.addAll(indicatorThresholdsMap.values());
         odd = !odd;
      }
      return hmcThresholdModels;
   }

   public List<String> getFilteringLawNames(Aircraft aircraft) {
      List<String> names = new ArrayList<>();
      lawNameToId.clear();
      try {
         for (FilteringLawDto dto : healthBusiness.getFilteringLaws(aircraft.getId())) {
            names.add(dto.getName());
            lawNameToId.put(dto.getName(), dto.getId());
         }
      } catch (Exception e) {
         LOGGER.warn("Could not load filtering laws", e);
      }
      names.sort(String::compareTo);
      return names;
   }

   public String getFilteringLawId(String name) {
      return lawNameToId.get(name);
   }

   public void saveDeviations() {
      List<IndicatorDeviationDto> newDeviations = new ArrayList<>();
      List<String> savedNames = new ArrayList<>();
      List<WCThresholdModel> savedModels = new ArrayList<>();
      Aircraft deviationAircraft = null;
      Date now = new Date();
      lastSavedDeviationIndicatorNames.clear();
      for (Entry<Aircraft, WCMechanicalComponentModel> entry : data.entrySet()) {
         deviationAircraft = entry.getKey();
         Multimap<Indicator, WCThresholdModel> thresholds = getComponentThresholds(entry.getValue());

         for (WCThresholdModel model : thresholds.values()) {
            LOGGER.info(" Value {} deviated={} deviationId={} icId={} te={}", model.getIndicator().getName(), model.isDeviated(), model.getDeviationId(),
model.getIndicatorConfigurationId(),
                  model.getTeReference());
            /*
             * if (!model.isDeviated() || model.getDeviationId() != null) { continue; }
             */

            if (!model.isDeviated() || model.getDeviationId() != null) {
               continue;
            }
            if (!hasDeviationFieldChanged(model)) {
               continue;
            }

            newDeviations.add(toDeviationDto(model, now));
            savedModels.add(model);

            // BCR-6605 remember the indicator for the "Deviation created" popup
            String indicatorName = getIndicatorDisplayName(model.getIndicator().getId());
            if (indicatorName.isEmpty()) {
               indicatorName = model.getIndicator().getName();
            }
            if (!savedNames.contains(indicatorName)) {
               savedNames.add(indicatorName);
            }
         }
      }
      if (!newDeviations.isEmpty() && deviationAircraft != null) {
         try {
            healthBusiness.createDeviations(deviationAircraft.getId(), MgsSession.getInstance().getUser().getLoginName(), newDeviations);
            // Only announce what the server actually accepted
            lastSavedDeviationIndicatorNames.addAll(savedNames);
            // BCR-6605 mark them saved so a second Apply does not create them again
            for (WCThresholdModel saved : savedModels) {
               saved.setDeviationId("SAVED");
            }
         } catch (Exception e) {
            LOGGER.error("Could not save deviations", e);
         }
      }

   }

   private boolean hasDeviationFieldChanged(final WCThresholdModel model) {
      final WCThresholdBaseline baseline = model.getBaseline();
      if (baseline == null) {
         return false;
      }
      return !Objects.equals(baseline.getLevel(), model.getLevel())
            || !Objects.equals(baseline.getType(), model.getType())
            || !Objects.equals(baseline.getMinValue(), model.getMinValue())
            || !Objects.equals(baseline.getMaxValue(), model.getMaxValue())
            || !Objects.equals(baseline.getFilteringLawName(), model.getFilteringLawName())
            || !Objects.equals(baseline.getMaxPointsPerHour(), model.getMaxPointsPerHour())
            || (model.getTeReference() != null && !model.getTeReference().trim().isEmpty());
   }

   private IndicatorDeviationDto toDeviationDto(WCThresholdModel model, Date now) {
      IndicatorDeviationDto dto = new IndicatorDeviationDto();
      dto.setIndicatorConfigurationId(model.getIndicatorConfigurationId());
```

```
      dto.setStartDate(now);
      dto.setSeverity(model.getSeverity().name());
      dto.setTeReference(model.getTeReference());
      dto.setFilteringLawId(getFilteringLawId(model.getFilteringLawName()));
      WCThresholdBaseline baseline = model.getBaseline();
      if (baseline == null) {
         return dto;
      }
      if (!Objects.equals(baseline.getLevel(), model.getLevel())) {
         dto.setThresholdLevel(model.getLevel().name());
      }
      if (!Objects.equals(baseline.getType(), model.getType())) {
         dto.setThresholdType(model.getType().name());
      }
      if (!Objects.equals(baseline.getDefaultValue(), model.getDefaultValue())) {
         dto.setThresholdValue(model.getDefaultValue());
      }
      if (!Objects.equals(baseline.getMinValue(), model.getMinValue())) {
         dto.setMinValue(model.getMinValue());
      }
      if (!Objects.equals(baseline.getMaxValue(), model.getMaxValue())) {
         dto.setMaxValue(model.getMaxValue());
      }
      /*
       * if (!Objects.equals(baseline.getLearningTime(), model.getLearningTime())) { dto.setLearningTime(model.getLearningTime()); }
       */
      if (!Objects.equals(baseline.getWorkCardReference(), model.getWorkCardReference())) {
         dto.setWorkcard(model.getWorkCardReference());
      }
      if (!Objects.equals(baseline.getMaxPointsPerHour(), model.getMaxPointsPerHour())) {
         dto.setMaxPointsPerHour(model.getMaxPointsPerHour());
      }

      /*
       * if (!Objects.equals(baseline.getMinimumDeviation(), model.getMinimumDeviation())) { dto.setMinimumDeviation(model.getMinimumDeviation()); }
       */
      /*
       * if (!Objects.equals(baseline.getMeanFactor(), model.getMeanFactor())) { dto.setMeanFactor(model.getMeanFactor()); }
       */
      /*
       * if (!Objects.equals(baseline.getDeviationCoefficient(), model.getDeviationCoefficient())) { dto.setDeviationCoefficient(model.getDeviationCoefficient()); }
       */
      /*
       * if (!Objects.equals(baseline.getNbSuccessivePoints(), model.getNbSuccessivePoints())) { dto.setNbSuccessivePoints(model.getNbSuccessivePoints()); }
       */
      return dto;
   }

   public String getIndicatorDisplayName(String indicatorId) {
      String name = indicatorDisplayNames.get(indicatorId);
      return name != null ? name : "";
   }

   public List<IndicatorDeviationDto> findActiveDeviations(Aircraft aircraft) {
      if (aircraft == null) {
         return new ArrayList<>();
      }
      try {
         return healthBusiness.findActiveDeviations(aircraft.getId());
      } catch (Exception e) {
         LOGGER.warn("Could not load active deviations", e);
         return new ArrayList<>();
      }
   }

   public List<IndicatorDeviationDto> findDeviationHistory(Aircraft aircraft) {
      if (aircraft == null) {
         return new ArrayList<>();
      }
      try {
         return healthBusiness.findDeviationHistory(aircraft.getId());
      } catch (Exception e) {
         LOGGER.warn("Could not load deviation history", e);
         return new ArrayList<>();
      }
   }

   public void createFilteringLaw(Aircraft aircraft, FilteringLawDto dto) {
      try {
         healthBusiness.createFilteringLaw(aircraft.getId(), dto);
      } catch (Exception e) {
         LOGGER.error("Could not create filtering law", e);
      }
   }

   public void acknowledgeDeviations(Aircraft aircraft, List<String> deviationIds) {
      if (aircraft == null) {
         return;
      }
      try {
         healthBusiness.acknowledgeDeviations(aircraft.getId(), MgsSession.getInstance().getUser().getLoginName(), deviationIds, true);
      } catch (Exception e) {
         LOGGER.error("Could  not   acknowledge deviations", e);
      }
   }

   public List<FilteringLawDto> getFilteringLawDetails(Aircraft aircraft) {
      if (aircraft == null) {
         return new ArrayList<>();
      }
      try {
         return healthBusiness.getFilteringLaws(aircraft.getId());
      } catch (Exception e) {
         LOGGER.warn("Could not load filtering laws", e);
         return new ArrayList<>();
      }
   }

   public void updateFilteringLaw(FilteringLawDto dto) {
      try {
         healthBusiness.updateFilteringLaw(dto);
      } catch (Exception e) {
         LOGGER.error("Could not update filtering law", e);
      }
   }

   public void deleteFilteringLaw(Aircraft aircraft, String lawId) {
      try {
         healthBusiness.deleteFilteringLaw(aircraft.getId(), lawId);
      } catch (Exception e) {
         LOGGER.error("Could not delete filtering law", e);
      }
   }

   public List<String> getLastSavedDeviationIndicatorNames() {
      return lastSavedDeviationIndicatorNames;
   }

}
```

HealthCommonBusiness.java

Package: com.eurocopter.mgs.business.health.impl
Change: contextual-parameter loop moved inside its null guard

```
/**************************************/
/** Copyright Airbus Helicopters 2019 **/
/**************************************/
package com.eurocopter.mgs.business.health.impl;

import java.util.ArrayList;
import java.util.Date;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import java.util.Set;
import java.util.UUID;

import com.eurocopter.mgs.business.health.api.IHealthCommonBusiness;
import com.eurocopter.mgs.business.health.model.IndicatorModel;
import com.eurocopter.mgs.business.health.model.WCThresholdModel;
import com.eurocopter.mgs.business.parameter.ParameterManager;
import com.eurocopter.mgs.core.persistence.PersistenceManager;
import com.eurocopter.mgs.datamodel.aircraft.Aircraft;
import com.eurocopter.mgs.datamodel.alarm.Alarm;
import com.eurocopter.mgs.datamodel.flight.Flight;
import com.eurocopter.mgs.datamodel.flight.FlightIndicatorValue;
import com.eurocopter.mgs.datamodel.health.common.Accelerometer;
import com.eurocopter.mgs.datamodel.health.common.IAcquisition;
import com.eurocopter.mgs.datamodel.health.common.IndicatorValue;
import com.eurocopter.mgs.datamodel.health.monitoring.AircraftDisplayData;
import com.eurocopter.mgs.datamodel.health.monitoring.ContextualParameter;
import com.eurocopter.mgs.datamodel.health.monitoring.FilteringLaw;
import com.eurocopter.mgs.datamodel.health.monitoring.FixedThresholdConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.HealthAlarm;
import com.eurocopter.mgs.datamodel.health.monitoring.HealthMechanicalComponent;
import com.eurocopter.mgs.datamodel.health.monitoring.HealthMechanicalComponentConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.HealthMonitoringConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.Indicator;
import com.eurocopter.mgs.datamodel.health.monitoring.IndicatorConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.IndicatorDeviation;
import com.eurocopter.mgs.datamodel.health.monitoring.IndicatorDisplay;
import com.eurocopter.mgs.datamodel.health.monitoring.IndicatorMarker;
import com.eurocopter.mgs.datamodel.health.monitoring.LearnedThresholdConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.ThresholdConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.ThresholdType;
import com.eurocopter.mgs.datamodel.systemstatus.IndicatorInvalidity;
import com.eurocopter.mgs.datamodel.systemstatus.SensorAnomaly;
import com.eurocopter.mgs.service.exception.ConcurrentAccessException;
import com.eurocopter.mgs.service.exception.EntityNotFoundException;
import com.eurocopter.mgs.service.health.common.api.IHealthAlarmService;
import com.eurocopter.mgs.service.health.common.api.IHealthCommonService;
import com.eurocopter.mgs.service.health.common.dto.AlarmIndicatorModel;
import com.eurocopter.mgs.service.health.common.dto.ContextualParameterDto;
import com.eurocopter.mgs.service.health.common.dto.FilteringLawDto;
import com.eurocopter.mgs.service.health.common.dto.HealthAcknowledgementDto;
import com.eurocopter.mgs.service.health.common.dto.IndicatorDeviationDto;
import com.eurocopter.mgs.service.marker.dto.BatchIndicatorMarkerDto;
import com.eurocopter.mgs.service.marker.dto.IndicatorMarkerDto;
import com.eurocopter.mgs.service.user.api.IUserService;
import com.eurocopter.mgs.xml.configuration.unit.Unit;

public class HealthCommonBusiness implements IHealthCommonBusiness {

   private final IHealthCommonService healthCommonService;

   private final IHealthAlarmService healthAlarmService;

   private final IUserService userService;

   private PersistenceManager persistenceManager;

   private ParameterManager parameterManager;

   private static final String RELEARN_DESCRIPTION_CREATE = "New  deviation   -  ";

   private static final String RELEARN_DESCRIPTION_DELETE = "Deviation  deleted  -  ";

   public HealthCommonBusiness(IHealthCommonService healthCommonService, IHealthAlarmService healthAlarmService, IUserService userService) {
      this.healthCommonService = healthCommonService;
      this.healthAlarmService = healthAlarmService;
      this.userService = userService;
   }

   public void setPersistenceManager(PersistenceManager persistenceManager) {
      this.persistenceManager = persistenceManager;
   }

   public void setParameterManager(ParameterManager parameterManager) {
      this.parameterManager = parameterManager;
   }

   @Override
   public List<Accelerometer> getAccelerometers(String aircraftId) throws EntityNotFoundException {
      Aircraft aircraft = persistenceManager.getEntityManager().find(Aircraft.class, aircraftId);
      if (aircraft == null) {
         throw new EntityNotFoundException("Aircraft not found [id = " + aircraftId + "]");
      }
      return new ArrayList<>(aircraft.getAccelerometers());
   }

   @Override
   public AircraftDisplayData getAircraftDisplayData(String aircraftId) {
      return healthCommonService.getAircraftDisplayData(aircraftId);
   }

   @Override
   public List<HealthMechanicalComponent> getHealthMechanicalComponents(String aircraftId) {
      return healthCommonService.getHealthMechanicalComponents(aircraftId);
   }

   @Override
   public Map<Indicator, IndicatorModel> getIndicatorModels(String userId, String aircraftId) throws EntityNotFoundException {
      Map<Indicator, HealthMechanicalComponent> indicatorMechanicalComponents = new HashMap<>();

      Map<Indicator, IndicatorDisplay> resultMap = new HashMap<>();
      List<HealthMechanicalComponent> healthMechanicalComponents = healthCommonService.getHealthMechanicalComponents(aircraftId);
      for (HealthMechanicalComponent comp : healthMechanicalComponents) {
         for (Indicator indicator : comp.getIndicators().values()) {
            indicatorMechanicalComponents.put(indicator, comp);
            resultMap.put(indicator, indicator.getDisplay());
         }
      }

      resultMap.putAll(userService.getIndicatorDisplays(userId, aircraftId));

      // Load the health monitoring configuration
      HealthMonitoringConfiguration healthConfig = getLatestConfiguration(aircraftId);
      Map<Indicator, IndicatorModel> indicatorModels = new HashMap<>(resultMap.size());
      for (Entry<Indicator, IndicatorDisplay> entry : resultMap.entrySet()) {
         Indicator indicator = entry.getKey();
         IndicatorDisplay indicatorDisplay = entry.getValue();
         if (indicatorMechanicalComponents.get(indicator) != null) {
            Object[] availableUnit = parameterManager.getUnitDefinitions().getPhysicalQuantityFor(indicatorDisplay.getUnitName()).getAllUnits().toArray(new Unit[0]);
            WCThresholdModel[] thresholdModels = getThresholds(indicatorMechanicalComponents.get(indicator), indicator, healthConfig);
            indicatorModels.put(entry.getKey(), new IndicatorModel(indicator, indicatorDisplay, availableUnit, thresholdModels));
```

```
        }
    }

    return indicatorModels;
}

@Override
public IndicatorConfiguration getLatestIndicatorConfiguration(String aircraftId, String indicatorId) throws EntityNotFoundException {
    HealthMonitoringConfiguration healthConfiguration = getLatestConfiguration(aircraftId);
    return getIndicatorConfiguration(indicatorId, healthConfiguration);
}

@Override
public IndicatorConfiguration getIndicatorConfigurationForFlight(String flightId, String indicatorId) throws EntityNotFoundException {
    HealthMonitoringConfiguration healthconfiguration = healthCommonService.getConfigurationForFlight(flightId);
    return getIndicatorConfiguration(indicatorId, healthconfiguration);
}

@Override
public FlightIndicatorValue getFlightIndicatorValue(String flightId, String indicatorId) throws EntityNotFoundException {
    return healthCommonService.getFlightIndicatorValue(flightId, indicatorId);
}

@Override
public void updateCloseMonitoringStatus(String aircraftId, Set<String> kept, Set<String> withdrawn) {
    healthAlarmService.updateCloseMonitoringStatus(aircraftId, kept, withdrawn);
}

@Override
public Map<String, Object> getHealthParameters() {
    return healthCommonService.getHealthParameters();
}

@Override
public HealthMonitoringConfiguration getLatestConfiguration(String aircraftId) throws EntityNotFoundException {
    return healthCommonService.getLatestConfiguration(aircraftId);
}

@Override
public Map<HealthMechanicalComponent, HealthMechanicalComponentConfiguration> getActiveComponentsAndConfiguration(String aircraftId) {
    return healthCommonService.getActiveComponentsAndConfiguration(aircraftId);
}

@Override
public Map<String, Map<IndicatorValue, IAcquisition>> getIndicatorValues(String avionicName, String aircraftId, String indicatorId) {
    return healthCommonService.getIndicatorValues(avionicName, aircraftId, indicatorId);
}

@Override
public void createRelearnThresholdRequests(String aircraftId, List<String> learnedThresholdIds, long historyLearningTime,
        String learningReason, String loginName) throws ConcurrentAccessException {
    healthCommonService.createRelearnThresholdRequests(aircraftId, learnedThresholdIds, historyLearningTime, learningReason, loginName);
}

@Override
public Map<HealthAlarm, Flight> getNotAckHealthAlarms(String aircraftId) {
    Map<HealthAlarm, Flight> map = new HashMap<>();
    for (AlarmIndicatorModel healthAlarmModel : getNotAckHealthAlarmModels(aircraftId)) {
        map.put((HealthAlarm) healthAlarmModel.getAlarmIndicator(), healthAlarmModel.getFlight());
    }
    return map;
}

@Override
public Set<AlarmIndicatorModel> getNotAckHealthAlarmModels(String aircraftId) {
    return healthAlarmService.getNotAckHealthAlarms(aircraftId);
}

@Override
public Map<HealthAlarm, Flight> getHealthAlarmsByIndicator(String indicatorId, Boolean acknowledged) {
    return healthAlarmService.getHealthAlarmsByIndicator(indicatorId, acknowledged);
}

@Override
public Map<SensorAnomaly, Flight> getSensorAnomaliesByIndicator(String indicatorId) {
    return healthAlarmService.getSensorAnomaliesByIndicator(indicatorId);
}

@Override
public Map<IndicatorInvalidity, Flight> getIndicatorInvaliditiesByIndicator(String indicatorId) {
    return healthAlarmService.getIndicatorInvaliditiesByIndicator(indicatorId);
}

@Override
public Map<Alarm, Flight> getNotAcknowledgedHealthAlarmsByHmc(String componentId) {
    return healthAlarmService.getNotAcknowledgedHealthAlarmsByHmc(componentId);
}

@Override
public Map<? extends Alarm, Flight> getAcknowledgedGreyHealthAlarmsAndVpuAnomaliesByHmc(String componentId) {
    Map<Alarm, Flight> result = new HashMap<>();
    result.putAll(healthAlarmService.getAcknowledgedHealthGreyAlarmsByHmc(componentId));
    result.putAll(healthAlarmService.getAcknowledgedVpuAnomaliesByHmc(componentId));
    return result;
}

@Override
public void acknowledgeAircraftHealthAlarms(String aircraftId, HealthAcknowledgementDto paramDto, String login) throws ConcurrentAccessException {
    healthAlarmService.acknowledgeAircraftHealthAlarms(aircraftId, paramDto, login);
}

@Override
public void acknowledgeMechanicalComponentAlarms(String aircraftId, String mcPath, HealthAcknowledgementDto paramDto, String login)
        throws EntityNotFoundException, ConcurrentAccessException {
    healthAlarmService.acknowledgeMechanicalComponentAlarms(aircraftId, mcPath, paramDto, login);
}

@Override
public void acknowledgeIndicatorAlarms(String aircraftId, String indicatorId, HealthAcknowledgementDto paramDto, String login)
        throws EntityNotFoundException, ConcurrentAccessException {
    healthAlarmService.acknowledgeIndicatorAlarms(aircraftId, indicatorId, paramDto, login);
}

@Override
public void closeMonitorAircraftHealthAlarms(String aircraftId, HealthAcknowledgementDto paramDto, String login) throws ConcurrentAccessException {
    healthAlarmService.closeMonitorAircraftHealthAlarms(aircraftId, paramDto, login);
}

@Override
public void closeMonitorMechanicalComponentAlarms(String aircraftId, String mcPath, HealthAcknowledgementDto paramDto, String login) throws ConcurrentAccessException {
    healthAlarmService.closeMonitorMechanicalComponentAlarms(aircraftId, mcPath, paramDto, login);
}

@Override
public void closeMonitorIndicatorAlarms(String aircraftId, String indicatorId, HealthAcknowledgementDto paramDto, String login) throws ConcurrentAccessException {
    healthAlarmService.closeMonitorIndicatorAlarms(aircraftId, indicatorId, paramDto, login);
}

@Override
public List<LearnedThresholdConfiguration> getIndicatorLatestLearnedThresholds(String aircraftId, String indicatorId) throws EntityNotFoundException {
    return healthCommonService.getIndicatorLatestLearnedThresholds(aircraftId, indicatorId);
}

@Override
public boolean hasLearnedThresholds(String aircraftId, String indicatorId) throws EntityNotFoundException {
    return healthCommonService.hasLearnedThresholds(aircraftId, indicatorId);
}

@Override
public Map<IndicatorMarker, String> getIndicatorMarkers(String aircraftId, String indicatorId) {
    return healthCommonService.getIndicatorMarkers(aircraftId, indicatorId);
}
```

```
   @Override
   public void createIndicatorMarker(IndicatorMarkerDto indicatorMarkerDto) throws EntityNotFoundException {
      healthCommonService.createIndicatorMarker(indicatorMarkerDto);
   }

   @Override
   public void createBatchIndicatorMarkers(BatchIndicatorMarkerDto batchIndicatorMarkerDto) throws EntityNotFoundException {
      healthCommonService.createBatchIndicatorMarkers(batchIndicatorMarkerDto);
   }

   @Override
   public void removeIndicatorMarker(String flightIndicatorValueId, String markerId) throws EntityNotFoundException {
      healthCommonService.removeIndicatorMarker(flightIndicatorValueId, markerId);
   }

   @Override
   public void removeAcknowledgementMarker(String alarmId) throws EntityNotFoundException {
      healthCommonService.removeAcknowledgementMarker(alarmId);
   }

   @Override
   public Map<String, HealthMechanicalComponentConfiguration> getMechanicalComponentConfigurationsForAircraft(String aircraftId, String mechanicalComponentId) throws
EntityNotFoundException {
      return healthCommonService.getMechanicalComponentConfigurationsForAircraft(aircraftId, mechanicalComponentId);
   }

   /**
    * Gets the latest thresholds for the indicator
    */
   private WCThresholdModel[] getThresholds(HealthMechanicalComponent mechanicalComponent, Indicator indicator, HealthMonitoringConfiguration healthMonitoringConfiguration) {
      Set<ThresholdConfiguration> latestThresholdConfigs;
      IndicatorConfiguration indicatorConfig =
healthMonitoringConfiguration.getMechanicalComponentConfigurations().get(mechanicalComponent).getIndicatorConfigurations().get(indicator);
      if (indicatorConfig != null) {
         latestThresholdConfigs = indicatorConfig.getThresholdConfigurations();
      } else {
         latestThresholdConfigs = new HashSet<>();
      }

      List<WCThresholdModel> existingThresholdModels = new ArrayList<>();

      for (ThresholdConfiguration existingThresholdConfig : latestThresholdConfigs) {

         WCThresholdModel thresholdModel = new WCThresholdModel(indicator, mechanicalComponent.getFunctionalId());
         existingThresholdModels.add(thresholdModel);

         // update data with the existing threshold values
         thresholdModel.setSeverity(existingThresholdConfig.getThreshold().getSeverity());
         thresholdModel.setEnabled(Boolean.TRUE);
         thresholdModel.setLevel(existingThresholdConfig.getThresholdLevel());
         thresholdModel.setWorkCardReference(existingThresholdConfig.getWorkcard());
         thresholdModel.setNbSuccessivePoints(existingThresholdConfig.getNbSuccessivePoints());
         thresholdModel.setMinimumEngineOilTemperature(existingThresholdConfig.getMinimumEngineOilTemperature());

         if (existingThresholdConfig instanceof LearnedThresholdConfiguration learnedConfiguration) {
            thresholdModel.setType(ThresholdType.LEARNED);
            // default value
            thresholdModel.setDefaultValue(learnedConfiguration.getValue() != null ? learnedConfiguration.getValue() : null);
            // max value
            thresholdModel.setMaxValue(Double.valueOf(learnedConfiguration.getMaximumThreshold()));
            // min value
            if (learnedConfiguration.getMinimumThreshold() != null) {
               thresholdModel.setMinValue(learnedConfiguration.getMinimumThreshold());
            }
            // learning time
            thresholdModel.setLearningTime(learnedConfiguration.getLearningTime());
            thresholdModel.setDeviationCoefficient(learnedConfiguration.getDeviationCoefficient());
            thresholdModel.setMeanFactor(learnedConfiguration.getMeanFactor());
            thresholdModel.setMinimumDeviation(learnedConfiguration.getMinimumDeviation());
         } else if (existingThresholdConfig instanceof FixedThresholdConfiguration fixedThreshold) {
            thresholdModel.setType(ThresholdType.FIXED);
            // default value
            thresholdModel.setDefaultValue(fixedThreshold.getValue());
         }
      }
      return existingThresholdModels.toArray(new WCThresholdModel[0]);
   }

   @Override
   public Accelerometer getAccelerometer(String indicatorId, String aircraftId) {
      return healthAlarmService.getAccelerometer(indicatorId, aircraftId);
   }

   @Override
   public Set<ThresholdConfiguration> getThresholdConfigurations(String thresholdId) {
      return healthCommonService.getThresholdConfigurations(thresholdId);
   }

   @Override
   public boolean hasReadIndicators(String aircraftId, String mechanicalComponentId) {
      return healthCommonService.hasReadIndicators(aircraftId, mechanicalComponentId);
   }

   @Override
   public List<IndicatorDeviationDto> findActiveDeviations(String aircraftId)
         throws EntityNotFoundException {
      return toDtos(healthCommonService.findActiveDeviations(getIndicatorIds(aircraftId)));
   }

   @Override
   public List<IndicatorDeviationDto> findDeviationHistory(String aircraftId)
         throws EntityNotFoundException {
      return toDtos(healthCommonService.findDeviationHistory(getIndicatorIds(aircraftId)));
   }

   private IndicatorConfiguration getIndicatorConfiguration(String indicatorId, HealthMonitoringConfiguration healthConfiguration) throws EntityNotFoundException {
      for (Map.Entry<HealthMechanicalComponent, HealthMechanicalComponentConfiguration> mcEntry : healthConfiguration.getMechanicalComponentConfigurations().entrySet()) {
         for (Map.Entry<Indicator, IndicatorConfiguration> indicatorEntry : mcEntry.getValue().getIndicatorConfigurations().entrySet()) {
            if (indicatorEntry.getKey().getId().equals(indicatorId)) {
               return indicatorEntry.getValue();
            }
         }
      }
      throw new EntityNotFoundException("Missing indicator configuration");
   }

   /*
    * @Override public List<FilteringLawDto> getFilteringLaws(String aircraftId) throws EntityNotFoundException { List<FilteringLawDto> result = new ArrayList<>();
HealthMonitoringConfiguration
    * configuration = getLatestConfiguration(aircraftId); for (FilteringLaw law : configuration.getFilteringLaws()) { FilteringLawDto dto = new FilteringLawDto();
dto.setId(law.getId());
    * dto.setName(law.getName()); result.add(dto); } return result; }
    */

   /*
    * @Override public void createDeviations(List<IndicatorDeviationDto> dtos) { for (IndicatorDeviationDto dto : dtos) { IndicatorDeviation d = new
IndicatorDeviation(UUID.randomUUID().toString());
    * d.setIndicatorConfiguration(healthCommonService.findIndicatorConfiguration(dto.getIndicatorConfigurationId()));
d.setFilteringLaw(healthCommonService.findFilteringLaw(dto.getFilteringLawId()));
    * d.setTeReference(dto.getTeReference()); d.setStartDate(dto.getStartDate()); d.setEndDate(null); d.setSeverity(dto.getSeverity());
d.setThresholdLevel(dto.getThresholdLevel());
    * d.setThresholdType(dto.getThresholdType()); d.setThresholdValue(dto.getThresholdValue()); d.setMinValue(dto.getMinValue()); d.setMaxValue(dto.getMaxValue());
    * d.setLearningTime(dto.getLearningTime()); d.setMinimumDeviation(dto.getMinimumDeviation()); d.setMeanFactor(dto.getMeanFactor());
d.setDeviationCoefficient(dto.getDeviationCoefficient());
    * d.setNbSuccessivePoints(dto.getNbSuccessivePoints()); healthCommonService.createDeviation(d); } }
    */

   /*
    * @Override public void acknowledgeDeviations(List<String> deviationIds) { Date now = new Date(); for (String id : deviationIds) {
healthCommonService.acknowledgeDeviation(id, now); } }
    */
```

```
   @Override
   public void createDeviations(String aircraftId, String loginName, List<IndicatorDeviationDto> dtos) throws EntityNotFoundException, ConcurrentAccessException {
      for (IndicatorDeviationDto dto : dtos) {
         IndicatorConfiguration indicatorConfiguration = healthCommonService.findIndicatorConfiguration(dto.getIndicatorConfigurationId());
         IndicatorDeviation d = new IndicatorDeviation(UUID.randomUUID().toString());
         d.setIndicatorConfiguration(indicatorConfiguration);
         d.setFilteringLaw(healthCommonService.findFilteringLaw(dto.getFilteringLawId()));
         d.setTeReference(dto.getTeReference());
         d.setStartDate(dto.getStartDate());
         d.setEndDate(null);
         d.setSeverity(dto.getSeverity());
         d.setThresholdLevel(dto.getThresholdLevel());
         d.setThresholdType(dto.getThresholdType());
         d.setThresholdValue(dto.getThresholdValue());
         d.setMinValue(dto.getMinValue());
         d.setMaxValue(dto.getMaxValue());
         d.setLearningTime(dto.getLearningTime());
         d.setMinimumDeviation(dto.getMinimumDeviation());
         d.setMeanFactor(dto.getMeanFactor());
         d.setDeviationCoefficient(dto.getDeviationCoefficient());
         d.setNbSuccessivePoints(dto.getNbSuccessivePoints());
         d.setMaxPointsPerHour(dto.getMaxPointsPerHour());
         healthCommonService.createDeviation(d);
         launchRelearn(aircraftId, loginName, getLearnedThresholdIds(aircraftId, indicatorConfiguration.getIndicator().getId(), dto.getSeverity(), dto.getThresholdType()),
               RELEARN_DESCRIPTION_CREATE + dto.getTeReference());
      }
   }

   @Override
   public void acknowledgeDeviations(String aircraftId, String loginName, List<String> deviationIds, boolean launchRelearn) throws EntityNotFoundException,
ConcurrentAccessException {

      List<IndicatorDeviation> closing = new ArrayList<>();
      for (IndicatorDeviation deviation : healthCommonService.findActiveDeviations(getIndicatorIds(aircraftId))) {
         if (deviationIds.contains(deviation.getId())) {
            closing.add(deviation);
         }
      }
      Date now = new Date();
      for (String id : deviationIds) {
         healthCommonService.acknowledgeDeviation(id, now);
      }

      if (launchRelearn) {
         for (IndicatorDeviation deviation : closing) {
            launchRelearn(aircraftId, loginName, getLearnedThresholdIds(aircraftId, deviation.getIndicatorConfiguration().getIndicator().getId(), deviation.getSeverity(), null),
                  RELEARN_DESCRIPTION_DELETE + deviation.getTeReference());

         }
      }

   }

   private List<String> getLearnedThresholdIds(String aircraftId, String indicatorId, String severity,
         String deviatedType) throws EntityNotFoundException {
      List<String> ids = new ArrayList<>();
      if (deviatedType != null && !ThresholdType.LEARNED.name().equals(deviatedType)) {
         // No return Id;
      }

      for (LearnedThresholdConfiguration threshold : getIndicatorLatestLearnedThresholds(aircraftId, indicatorId)) {
         if (severity != null && !severity.equals(threshold.getThreshold().getSeverity().name())) {
            continue;
         }
         ids.add(threshold.getId());
      }
      return ids;
   }

   private void launchRelearn(String aircraftId, String loginName, List<String> thresholdIds, String description) throws ConcurrentAccessException {
      if (thresholdIds.isEmpty()) {
         return;
      }
      createRelearnThresholdRequests(aircraftId, thresholdIds, 0L, description, loginName);
   }

   private List<IndicatorDeviationDto> toDtos(List<IndicatorDeviation> deviations) {
      List<IndicatorDeviationDto> result = new ArrayList<>();
      for (IndicatorDeviation d : deviations) {
         IndicatorDeviationDto dto = new IndicatorDeviationDto();
         dto.setId(d.getId());
         dto.setIndicatorConfigurationId(d.getIndicatorConfiguration().getId());
         dto.setIndicatorId(d.getIndicatorConfiguration().getIndicator().getId());
         StringBuilder params = new StringBuilder();
         if (d.getFilteringLaw() != null) {
            dto.setFilteringLawId(d.getFilteringLaw().getId());
            dto.setFilteringLawName(d.getFilteringLaw().getName());
            // BCR-6605 this loop must stay inside the null check
            for (ContextualParameter p : d.getFilteringLaw().getContextualParameters()) {
               if (params.length() > 0) {
                  params.append("  |  ");
               }
               params.append(p.getName()).append(", min=").append(p.getMinValue()).append(", max=").append(p.getMaxValue());
               if (p.getThresholdOfVariabilityValue() != null) {
                  params.append(", S=").append(p.getThresholdOfVariabilityValue());
               }
            }
         }
         dto.setFilteringLawParameters(params.toString());
         dto.setTeReference(d.getTeReference());
         dto.setStartDate(d.getStartDate());
         dto.setEndDate(d.getEndDate());
         dto.setSeverity(d.getSeverity());
         dto.setThresholdLevel(d.getThresholdLevel());
         dto.setThresholdType(d.getThresholdType());
         dto.setThresholdValue(d.getThresholdValue());
         dto.setMinValue(d.getMinValue());
         dto.setMaxValue(d.getMaxValue());
         dto.setLearningTime(d.getLearningTime());
         dto.setMinimumDeviation(d.getMinimumDeviation());
         dto.setMeanFactor(d.getMeanFactor());
         dto.setDeviationCoefficient(d.getDeviationCoefficient());
         dto.setNbSuccessivePoints(d.getNbSuccessivePoints());
         dto.setMaxPointsPerHour(d.getMaxPointsPerHour());
         result.add(dto);
      }
      return result;
   }

   /*
    * private List<String> getIndicatorConfigurationIds(String aircraftId) throws EntityNotFoundException { List<String> ids = new ArrayList<>(); HealthMonitoringConfiguration
configuration =
    * getLatestConfiguration(aircraftId); for (HealthMechanicalComponentConfiguration mcc : configuration.getMechanicalComponentConfigurations().values()) { for
(IndicatorConfiguration ic :
    * mcc.getIndicatorConfigurations().values()) { ids.add(ic.getId()); } } return ids; }
    */

   private List<String> getIndicatorIds(String aircraftId) throws EntityNotFoundException {
      List<String> ids = new ArrayList<>();
      HealthMonitoringConfiguration configuration = getLatestConfiguration(aircraftId);
      for (HealthMechanicalComponentConfiguration mcc : configuration.getMechanicalComponentConfigurations().values()) {
         for (IndicatorConfiguration ic : mcc.getIndicatorConfigurations().values()) {
            ids.add(ic.getIndicator().getId());
         }
      }
      return ids;
   }

   @Override
   public List<FilteringLawDto> getFilteringLaws(String aircraftId) throws EntityNotFoundException {
      List<FilteringLawDto> result = new ArrayList<>();
```

```
      for (FilteringLaw law : healthCommonService.findFilteringLaws(aircraftId)) {
         FilteringLawDto dto = new FilteringLawDto();
         dto.setId(law.getId());
         dto.setName(law.getName());
         dto.setOriginal(law.isOriginal());
         dto.setInUse(healthCommonService.isFilteringLawUsedByConfiguration(aircraftId, law.getName()) || healthCommonService.isFilteringLawUsedByDeviation(law.getId()));
         for (ContextualParameter p : law.getContextualParameters()) {
            ContextualParameterDto pDto = new ContextualParameterDto();
            pDto.setId(p.getId());
            pDto.setName(p.getName());
            pDto.setMinValue(p.getMinValue());
            pDto.setMaxValue(p.getMaxValue());
            pDto.setThresholdOfVariabilityValue(p.getThresholdOfVariabilityValue());
            dto.getParameters().add(pDto);
         }
         result.add(dto);
      }
      result.sort((a, b) -> a.getName().compareTo(b.getName()));
      return result;
   }

   /*
    * @Override public void createFilteringLaw(String aircraftId, FilteringLawDto dto) throws EntityNotFoundException { FilteringLaw law = new
FilteringLaw(UUID.randomUUID().toString());
    * law.setName(dto.getName()); law.setOriginal(false); applyParameters(law, dto); healthCommonService.createFilteringLaw(aircraftId, law); }
    */

   /*
    * @Override public void updateFilteringLaw(FilteringLawDto dto) { FilteringLaw law = healthCommonService.findFilteringLawById(dto.getId()); if (law == null) { return; }
law.setName(dto.getName());
    * law.getContextualParameters().clear(); applyParameters(law, dto); }
    */

   @Override
   public void deleteFilteringLaw(String aircraftId, String lawId) throws EntityNotFoundException {
      healthCommonService.deleteFilteringLaw(aircraftId, lawId);
   }

   /*
    * private void applyParameters(FilteringLaw law, FilteringLawDto dto) { for (ContextualParameterDto pDto : dto.getParameters()) { ContextualParameter p = new
    * ContextualParameter(UUID.randomUUID().toString()); p.setName(pDto.getName()); p.setMinValue(pDto.getMinValue()); p.setMaxValue(pDto.getMaxValue());
    * p.setThresholdOfVariabilityValue(pDto.getThresholdOfVariabilityValue()); law.getContextualParameters().add(p); } }
    */

   @Override
   public void createFilteringLaw(String aircraftId, FilteringLawDto dto)
         throws EntityNotFoundException {
      healthCommonService.createFilteringLaw(aircraftId, dto.getName(), dto.getParameters());
   }

   @Override
   public void updateFilteringLaw(FilteringLawDto dto) {
      healthCommonService.updateFilteringLaw(dto.getId(), dto.getName(), dto.getParameters());
   }

   @Override
   public boolean hasActiveDeviationOnThresholds(String aircraftId, List<String> thresholdIds) throws EntityNotFoundException {
      if (thresholdIds == null || thresholdIds.isEmpty()) {
         return false;
      }
      // The indicators those thresholds belong to
      Set<String> indicatorIds = new HashSet<>();
      HealthMonitoringConfiguration configuration = getLatestConfiguration(aircraftId);
      for (HealthMechanicalComponentConfiguration mcc : configuration.getMechanicalComponentConfigurations().values()) {
         for (IndicatorConfiguration ic : mcc.getIndicatorConfigurations().values()) {
            for (ThresholdConfiguration threshold : ic.getThresholdConfigurations()) {
               if (thresholdIds.contains(threshold.getId())) {
                  indicatorIds.add(ic.getIndicator().getId());
               }
            }
         }
      }
      if (indicatorIds.isEmpty()) {
         return false;
      }
      for (IndicatorDeviationDto dto : findActiveDeviations(aircraftId)) {
         if (indicatorIds.contains(dto.getIndicatorId())) {
            return true;
         }
      }
      return false;
   }
}
```

WCTreeCellModifier.java

Package: `com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.editor`
Change: Points cascade removed - each severity row keeps its own value

```
/**************************************/
/** Copyright Airbus Helicopters 2019 **/
/**************************************/
package com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.editor;

import java.text.NumberFormat;
import java.text.ParseException;
import java.util.Locale;

import org.eclipse.jface.dialogs.MessageDialog;
import org.eclipse.jface.viewers.ICellModifier;
import org.eclipse.jface.viewers.TreeViewer;
import org.eclipse.swt.widgets.Item;

import com.eurocopter.mgs.business.health.model.WCThresholdModel;
import com.eurocopter.mgs.core.presentation.i18n.Messages;
import com.eurocopter.mgs.datamodel.aircraft.Aircraft;
import com.eurocopter.mgs.datamodel.health.monitoring.ThresholdLevel;
import com.eurocopter.mgs.datamodel.health.monitoring.ThresholdType;
import com.eurocopter.mgs.presentation.common.MgsSession;
import com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.WCHealthMechanicalComponentModel;
import com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.WCMechanicalComponentModel;
import com.eurocopter.mgs.presentation.configuration.preferences.healthalarms.WCModelDisplayProperty;
import com.eurocopter.mgs.presentation.configuration.store.MgsPreferenceStoreManager;

public class WCTreeCellModifier implements ICellModifier {

   /** point character. */
   private static final String POINT = ".";

   /** Coma character. */
   private static final String COMA = ",";

   /** The format number. */
   private final NumberFormat format = NumberFormat.getInstance(Locale.getDefault());

   private final TreeViewer treeViewer;

   /**
    * Default constructor. Builds a WCTreeCellModifier.
    *
    * @param treeViewer
    *           TreeViewer treeViewer
    */
   public WCTreeCellModifier(TreeViewer treeViewer) {
      super();
      this.treeViewer = treeViewer;
      format.setMaximumFractionDigits(5);
   }

   @Override
   public boolean canModify(Object element, String property) {
      if (element instanceof WCMechanicalComponentModel) {
         if (WCModelDisplayProperty.MAX_TIME_WO_ACQ.equals(WCModelDisplayProperty.fromKey(property))) {
            return true;
         }
      }
      if (element instanceof WCHealthMechanicalComponentModel wcModel) {
         if (!wcModel.getThresholdModels().isEmpty()
               && WCModelDisplayProperty.WORKCARD_REF == WCModelDisplayProperty.fromKey(property)) {
            return MgsSession.getInstance().getUser().getProfile().canEditWorkcards();
         }
         if (WCModelDisplayProperty.MAX_TIME_WO_ACQ.equals(WCModelDisplayProperty.fromKey(property))) {
            return true;
         }
      }
      if (element instanceof WCThresholdModel thresholdModel) {
         if (thresholdModel.isDeviated()) {
            return false;
         }
         switch (WCModelDisplayProperty.fromKey(property)) {
            case COMPONENT:
               return false;
            case MIN_VALUE:
            case MAX_VALUE:
            case LEARN_TIME:
            case MIN_DEV:
            case MEAN_MULT_FACTOR:
            case DEV_COEF:
               // These values can only be modified if threshold is Learning
               WCThresholdModel wcThresholdModel = (WCThresholdModel) element;
               return ThresholdType.LEARNED.equals(wcThresholdModel.getType());
            case WORKCARD_REF:
               // The workcards column can only be modified by authorized users
               return MgsSession.getInstance().getUser().getProfile().canEditWorkcards();
            default:
               return true;
         }
      }
      return false;
   }

   @Override
   public Object getValue(Object element, String property) {
      if (element instanceof WCMechanicalComponentModel wcMechanicalComponent) {
         switch (WCModelDisplayProperty.fromKey(property)) {
            case COMPONENT:
               return wcMechanicalComponent.getName();
            case MAX_TIME_WO_ACQ:
               return getMaxTimeWOAcquition(wcMechanicalComponent);
            default:
               return "";
         }
      } else if (element instanceof WCHealthMechanicalComponentModel mcModel) {
         switch (WCModelDisplayProperty.fromKey(property)) {
            case WORKCARD_REF:
               return mcModel.getWorkcardReference() != null ? mcModel.getWorkcardReference() : "";
            case MAX_TIME_WO_ACQ:
               return format.format(mcModel.getMaxtimeWithoutAcquitision());
            default:
               return "";
         }
      } else if (element instanceof WCThresholdModel thresholdModel) {
         switch (WCModelDisplayProperty.fromKey(property)) {
            /*
             * BCR-6605 Check box code disable. case CHECKBOX: return thresholdModel.isEnabled();
             */

            case LEVEL:
               return thresholdModel.getLevel();

            case TYPE:
               return thresholdModel.getType();

            case VALUE:
               if (thresholdModel.getDefaultValue() != null) {
                  return format.format(thresholdModel.getDefaultValue());
               }
               return "";

            case MIN_VALUE:
               if (thresholdModel.getMinValue() != null) {
                  return format.format(thresholdModel.getMinValue());
```

```
            }
            return "";

         case MAX_VALUE:
            return format.format(thresholdModel.getMaxValue());

         case LEARN_TIME:
            return format.format(thresholdModel.getLearningTime() / 3600000.0);

         case WORKCARD_REF:
            return thresholdModel.getWorkCardReference();

         case MIN_DEV:
            if (thresholdModel.getMinimumDeviation() != null) {
               return format.format(thresholdModel.getMinimumDeviation());
            }
            return "";

         case MEAN_MULT_FACTOR:
            if (thresholdModel.getMeanFactor() != null) {
               return format.format(thresholdModel.getMeanFactor());
            }
            return "";

         case DEV_COEF:
            if (thresholdModel.getDeviationCoefficient() != null) {
               return format.format(thresholdModel.getDeviationCoefficient());
            }
            return "";

         case NB_POINTS:
            if (thresholdModel.getNbSuccessivePoints() != null) {
               return format.format(thresholdModel.getNbSuccessivePoints());
            }
            return "";
         case FILTERING_LAW:
            return thresholdModel.getFilteringLawName() != null ? thresholdModel.getFilteringLawName() : "";

         case TE:
            return thresholdModel.getTeReference() != null ? thresholdModel.getTeReference() : "";

         // BCR-6605 Points = IndicatorConfiguration.maxPointsPerHour
         case POINTS:
            if (thresholdModel.getMaxPointsPerHour() != null) {
               return format.format(thresholdModel.getMaxPointsPerHour());
            }
            return "";
         default:
            return "";

      }
   }
   return null;
}

@Override
public void modify(Object element, String property, Object value) {
   Object data;
   if (element instanceof Item) {
      data = ((Item) element).getData();

      try {
         if (data instanceof WCMechanicalComponentModel wcMechanicalComponent) {
            switch (WCModelDisplayProperty.fromKey(property)) {
               case MAX_TIME_WO_ACQ:
                  if (value != null && !value.toString().trim().isEmpty()
                        && !"none".equals(value.toString().trim())) {
                     long newValue = format.parse(replaceSeparatorValue(value).toString()).longValue();
                     if (newValue >= 0) {
                        updateCascadeMaxTimeWOAcq(wcMechanicalComponent, value,
                              wcMechanicalComponent.getName().contains("Engine"));
                     }
                  }
                  break;
               default:
                  break;
            }
            // BCR-6605 mechanical component field: configuration, keep marking dirty
            MgsPreferenceStoreManager.markAsDirty((Aircraft) treeViewer.getData("aircraft"));
            treeViewer.refresh();

         } else if (data instanceof WCHealthMechanicalComponentModel wCModel) {
            switch (WCModelDisplayProperty.fromKey(property)) {
               case WORKCARD_REF:
                  if (value != null && !value.toString().trim().isEmpty()) {
                     wCModel.setWorkCardReference(value.toString());
                  } else {
                     wCModel.setWorkCardReference("");
                  }
                  break;
               case MAX_TIME_WO_ACQ:
                  if (value != null && !value.toString().trim().isEmpty()
                        && !"none".equals(value.toString().trim())) {
                     long newValue = format.parse(replaceSeparatorValue(value).toString()).longValue();
                     if (wCModel.getMaxtimeWithoutAcquitision() != newValue && newValue >= 0) {
                        wCModel.setMaxtimeWithoutAcquitision(newValue);
                     }
                  } else
                     wCModel.setMaxtimeWithoutAcquitision(0);
                  break;
               default:
                  break;
            }
            // BCR-6605 mechanical component fields: configuration, keep marking dirty
            MgsPreferenceStoreManager.markAsDirty((Aircraft) treeViewer.getData("aircraft"));
            treeViewer.refresh();

         } else if (data instanceof WCThresholdModel thresholdModel) {
            boolean configurationField = false;
            switch (WCModelDisplayProperty.fromKey(property)) {
               /*
                * case CHECKBOX: thresholdModel.setEnabled((Boolean) value); break;
                */
               case LEVEL:
                  thresholdModel.setLevel((ThresholdLevel) value);
                  break;
               case TYPE:
                  thresholdModel.setType((ThresholdType) value);
                  break;
               case VALUE:
                  configurationField = true;
                  if (value != null && !value.toString().trim().isEmpty()
                        && !"none".equals(value.toString().trim())) {
                     Number val = format.parse(replaceSeparatorValue(value).toString());
                     thresholdModel.setDefaultValue(val.doubleValue());
                  } else {
                     thresholdModel.setDefaultValue(null);
                  }
                  break;
               case MIN_VALUE:
                  if (value != null && !value.toString().trim().isEmpty()) {
                     Number val = format.parse(replaceSeparatorValue(value).toString());
                     thresholdModel.setMinValue(val.doubleValue());
                  } else {
                     thresholdModel.setMinValue(null);
                  }
                  break;
               case MAX_VALUE:
                  if (value != null && !value.toString().trim().isEmpty()) {
                     Number val = format.parse(replaceSeparatorValue(value).toString());
                     thresholdModel.setMaxValue(val.doubleValue());
                  } else {
```

```
                  thresholdModel.setMaxValue(0.0d);
               }
               break;
            case LEARN_TIME:
               configurationField = true;
               if (value != null && !value.toString().trim().isEmpty()) {
                  long newValue = format.parse(replaceSeparatorValue(value).toString()).longValue() * 3600000;
                  if (thresholdModel.getLearningTime() != newValue && newValue >= 0) {
                     thresholdModel.setLearningTime(newValue);
                  }
               }
               break;
            case WORKCARD_REF:
               if (value != null && !value.toString().trim().isEmpty()) {
                  thresholdModel.setWorkCardReference(value.toString());
               } else {
                  thresholdModel.setWorkCardReference("");
               }
               break;
            case MIN_DEV:
               configurationField = true;
               if (value != null && !value.toString().trim().isEmpty()) {
                  Number val = format.parse(replaceSeparatorValue(value).toString());
                  thresholdModel.setMinimumDeviation(val.doubleValue());
               } else {
                  thresholdModel.setMinimumDeviation(null);
               }
               break;
            case MEAN_MULT_FACTOR:
               configurationField = true;
               if (value != null && !value.toString().trim().isEmpty()) {
                  Number val = format.parse(replaceSeparatorValue(value).toString());
                  thresholdModel.setMeanFactor(val.doubleValue());
               } else {
                  thresholdModel.setMeanFactor(null);
               }
               break;
            case DEV_COEF:
               configurationField = true;
               if (value != null && !value.toString().trim().isEmpty()) {
                  Number val = format.parse(replaceSeparatorValue(value).toString());
                  thresholdModel.setDeviationCoefficient(val.doubleValue());
               } else {
                  thresholdModel.setDeviationCoefficient(null);
               }
               break;
            case POINTS:
               // BCR-6605 deviation field. Each severity row keeps its own value: the deviation
               // table stores Points per severity, only the configuration is one per indicator.
               if (value != null && !value.toString().trim().isEmpty()) {
                  Number val = format.parse(replaceSeparatorValue(value).toString());
                  thresholdModel.setMaxPointsPerHour(val.intValue());
               } else {
                  thresholdModel.setMaxPointsPerHour(null);
               }
               break;
            case NB_POINTS:
               configurationField = true;
               if (value != null && !value.toString().trim().isEmpty()) {
                  Number val = format.parse(replaceSeparatorValue(value).toString());
                  thresholdModel.setNbSuccessivePoints(val.intValue());
               } else {
                  thresholdModel.setNbSuccessivePoints(null);
               }
               break;
            case FILTERING_LAW:
               if (value != null && !value.toString().trim().isEmpty()) {
                  thresholdModel.setFilteringLawName(value.toString());
               } else {
                  thresholdModel.setFilteringLawName(null);
               }
               break;
            case TE:
               if (value != null && !value.toString().trim().isEmpty()) {
                  thresholdModel.setTeReference(value.toString().trim());
               } else {
                  thresholdModel.setTeReference(null);
               }
               break;

            default:
               break;
            }
            // BCR-6605 only a configuration field creates a new A/C configuration
            if (configurationField) {
               MgsPreferenceStoreManager.markAsDirty((Aircraft) treeViewer.getData("aircraft"));
            }

            treeViewer.refresh();
         }
      } catch (ParseException e) {
         MessageDialog.openError(((Item) element).getDisplay().getActiveShell(),
               Messages.CORE.getString("healthAlarmPageDataInvalidTitle"),
               Messages.CORE.getMnemonicEscapedString("healthAlarmPageDataInvalidCellFormat"));
      }
   }
}

/**
 * Replace the separator value.
 *
 * @param value
 *           Object
 * @return the value
 */
private Object replaceSeparatorValue(final Object value) {
   Object val = value;
   Locale locale = Locale.getDefault();
   if (val.toString().contains(POINT) && (locale.equals(Locale.FRENCH) || locale.equals(Locale.FRANCE))) {
      val = val.toString().replace(POINT, COMA);
   } else if (val.toString().contains(COMA) && locale.equals(Locale.ENGLISH)) {
      val = val.toString().replace(COMA, POINT);
   }
   return val;
}

private void updateCascadeMaxTimeWOAcq(WCMechanicalComponentModel wcMechanicalComponent, Object value,
      boolean force) throws ParseException {

   for (WCHealthMechanicalComponentModel wCModel : wcMechanicalComponent.getHealthComponentModels()) {
      if (value != null && !value.toString().trim().isEmpty() && !"none".equals(value.toString().trim())) {
         Number val = format.parse(replaceSeparatorValue(value).toString());
         if (force || !wCModel.getHealthMechanicalComponent().getName().contains(":Engine"))
            wCModel.setMaxtimeWithoutAcquitision(val.longValue());
      }
   }
   for (WCMechanicalComponentModel wCModel : wcMechanicalComponent.getSubHealthWCModel()) {
      updateCascadeMaxTimeWOAcq(wCModel, value, force);
   }
}

/**
 * return the maximum time without acquisition if all the sub health component have the same value. Else return ""
 *
 * @return
 */
private String getMaxTimeWOAcquition(WCMechanicalComponentModel element) {

   String maxTimeWOAcqValue = null;
   for (WCHealthMechanicalComponentModel wCModel : element.getHealthComponentModels()) {
      if (maxTimeWOAcqValue == null) {
         maxTimeWOAcqValue = format.format(wCModel.getMaxtimeWithoutAcquitision());
```

```
            } else if (!maxTimeWOAcqValue.equals(format.format(wCModel.getMaxtimeWithoutAcquitision()))) {
               return "";
            }
         }
         for (WCMechanicalComponentModel wcModel : element.getSubHealthWCModel()) {
            if (maxTimeWOAcqValue == null) {
               maxTimeWOAcqValue = getMaxTimeWOAcquition(wcModel);
            } else if (!maxTimeWOAcqValue.equals(getMaxTimeWOAcquition(wcModel))) {
               return "";
            }
         }
         return maxTimeWOAcqValue;
      }

}
```

AbstractFilteringRuleAlgorithm.java

Package: `com.eurocopter.mgs.download.analysis.health.impl.filtering`
Change: deviated Points and deviated filtering law applied

```
package com.eurocopter.mgs.download.analysis.health.impl.filtering;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import java.util.Set;

import com.eurocopter.mgs.datamodel.alarm.Severity;
import com.eurocopter.mgs.datamodel.health.monitoring.FilteringLaw;
import com.eurocopter.mgs.datamodel.health.monitoring.HealthMonitoringConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.Indicator;
import com.eurocopter.mgs.datamodel.health.monitoring.IndicatorConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.IndicatorDeviation;
import com.eurocopter.mgs.download.analysis.health.algo.FilteringAlgorithm;
import com.eurocopter.mgs.download.analysis.health.data.IHealthIndicatorValueAcquisition;
import com.eurocopter.mgs.download.analysis.health.impl.HealthIndicatorValueAdapter;

public abstract class AbstractFilteringRuleAlgorithm<K> implements IFilteringRuleAlgorithm {

   private static final Comparator<HealthIndicatorValueAdapter> INDICATOR_VALUE_COMPARATOR = Comparator.comparing(HealthIndicatorValueAdapter::getAcquisitionDate);
   private final Map<K, Map<IndicatorConfiguration, List<HealthIndicatorValueAdapter>>> healthIndicatorValueAdapterByFileIdAndIndConf;

   /**
    * BCR-6605 the deviations applicable to the flight being analysed, keyed indicator.id + "|" + severity.
    * Empty unless setDeviations() was called, so filtering behaves exactly as before on any path that does
    * not set them.
    */
   private Map<String, IndicatorDeviation> deviations = Collections.emptyMap();

   public AbstractFilteringRuleAlgorithm() {
      this.healthIndicatorValueAdapterByFileIdAndIndConf = new HashMap<>();
   }

   /**
    * BCR-6605 give the algorithm the deviations that apply to this flight, so a deviated Points value
    * overrides the configured maxPointsPerHour.
    *
    * @param applicableDeviations
    *           keyed indicator.id + "|" + severity, may be null
    */
   public void setDeviations(final Map<String, IndicatorDeviation> applicableDeviations) {
      this.deviations = applicableDeviations != null ? applicableDeviations : Collections.emptyMap();
   }

   protected abstract boolean isFilteringLawApplicable(List<HealthIndicatorValueAdapter> indicatorValueAdapters);

   protected abstract K getKey(IHealthIndicatorValueAcquisition acquisition);

   protected abstract int getMaxPointsAtKey(K key, int configuredMaxPointsPerHour);

   protected abstract void applyFilteringLawAlgorithm(List<HealthIndicatorValueAdapter> values, Set<FilteringLaw> filteringLaws, String indicatorFilteringLaw);

   @Override
   public void recordAcquisition(IHealthIndicatorValueAcquisition acquisition, String indicatorId, Indicator indicator, IndicatorConfiguration indicatorConfiguration) {
      K key = getKey(acquisition);
      final Map<IndicatorConfiguration, List<HealthIndicatorValueAdapter>> indicatorsInFile = healthIndicatorValueAdapterByFileIdAndIndConf.computeIfAbsent(key, k -> new
HashMap<>());
      final List<HealthIndicatorValueAdapter> indicatorValues = indicatorsInFile.computeIfAbsent(indicatorConfiguration, k -> new ArrayList<>());
      indicatorValues.add(new HealthIndicatorValueAdapter(acquisition, indicatorConfiguration, indicatorId, indicator));
   }

   @Override
   public List<HealthIndicatorValueAdapter> getFilteredValues(HealthMonitoringConfiguration healthConfiguration) {
      for (Map<IndicatorConfiguration, List<HealthIndicatorValueAdapter>> indicators : healthIndicatorValueAdapterByFileIdAndIndConf.values()) {
         for (Entry<IndicatorConfiguration, List<HealthIndicatorValueAdapter>> entry2 : indicators.entrySet()) {
            final IndicatorConfiguration indicatorConfiguration = entry2.getKey();
            final Set<FilteringLaw> filteringLaws = healthConfiguration.getFilteringLaws();
            List<HealthIndicatorValueAdapter> indicatorValueAdapters = entry2.getValue();
            // BCR-6605 a deviation can replace the configured filtering law
            final String filteringLawName = getFilteringLawName(indicatorConfiguration);
            if (filteringLawName != null && isFilteringLawApplicable(indicatorValueAdapters)) {
               applyFilteringLawAlgorithm(indicatorValueAdapters, filteringLaws, filteringLawName);
            }
         }
      }

      final List<HealthIndicatorValueAdapter> valuesToSave = new ArrayList<>();
      for (Map.Entry<K, Map<IndicatorConfiguration, List<HealthIndicatorValueAdapter>>> indicatorsEntry : healthIndicatorValueAdapterByFileIdAndIndConf.entrySet()) {
         K key = indicatorsEntry.getKey();
         Map<IndicatorConfiguration, List<HealthIndicatorValueAdapter>> indicators = indicatorsEntry.getValue();
         for (Entry<IndicatorConfiguration, List<HealthIndicatorValueAdapter>> indicatorEntry : indicators.entrySet()) {
            final IndicatorConfiguration conf = indicatorEntry.getKey();
            final List<HealthIndicatorValueAdapter> healthIndicatorValueAdapterValues = indicatorEntry.getValue();
            // BCR-6605 a deviation on either severity of this indicator overrides the configured Points
            Integer configuredPoints = conf.getMaxPointsPerHour();
            final Integer deviatedPoints = getDeviatedPoints(conf);
            if (deviatedPoints != null) {
               configuredPoints = deviatedPoints;
            }
            if (configuredPoints == null) {
               valuesToSave.addAll(healthIndicatorValueAdapterValues);
            } else {
               healthIndicatorValueAdapterValues.sort(INDICATOR_VALUE_COMPARATOR);
               int maxPointsPerHour = getMaxPointsAtKey(key, configuredPoints);
               if (maxPointsPerHour > 0) {
                  valuesToSave.addAll(FilteringAlgorithm.filterHealthIndicatorValues(healthIndicatorValueAdapterValues, maxPointsPerHour));
               }
            }
         }
      }
      return valuesToSave;
   }

   /**
    * BCR-6605 Points is one value per indicator, but deviations are keyed per severity. Take the first
    * severity that actually carries a deviated value.
    *
    * @param conf
    *           the indicator configuration being filtered
    * @return the deviated Points, or null when no applicable deviation changed it
    */
   /**
    * BCR-6605 the filtering law to apply: the deviated one when a deviation set it, otherwise the
    * configured one.
    *
    * @param conf
    *           the indicator configuration being filtered
    * @return the law name to apply, or null when there is none
    */
   private String getFilteringLawName(final IndicatorConfiguration conf) {
      if (!deviations.isEmpty() && conf.getIndicator() != null) {
         for (Severity severity : Severity.values()) {
            IndicatorDeviation deviation = deviations.get(conf.getIndicator().getId() + "|" + severity.name());
            if (deviation != null && deviation.getFilteringLaw() != null) {
               return deviation.getFilteringLaw().getName();
            }
         }
      }
      return conf.getFilteringLaw();
```

```
    }

    private Integer getDeviatedPoints(final IndicatorConfiguration conf) {
        if (deviations.isEmpty() || conf.getIndicator() == null) {
            return null;
        }
        for (Severity severity : Severity.values()) {
            IndicatorDeviation deviation = deviations.get(conf.getIndicator().getId() + "|" + severity.name());
            if (deviation != null && deviation.getMaxPointsPerHour() != null) {
                return deviation.getMaxPointsPerHour();
            }
        }
        return null;
    }

}
```

HealthAcquisitionAnalyzer.java

Package: `com.eurocopter.mgs.download.analysis.health.impl`
Change: IHealthHistoryProvider threaded through; setDeviations wired

```
/**************************************/
/** Copyright Airbus Helicopters 2019 **/
/**************************************/
package com.eurocopter.mgs.download.analysis.health.impl;

import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.Date;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;
import java.util.NoSuchElementException;
import java.util.Set;
import java.util.UUID;

import org.apache.commons.collections4.MultiValuedMap;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

import com.eurocopter.mgs.datamodel.aircraft.Aircraft;
import com.eurocopter.mgs.datamodel.aircraft.Avionic;
import com.eurocopter.mgs.datamodel.flight.Flight;
import com.eurocopter.mgs.datamodel.flight.FlightIndicatorValue;
import com.eurocopter.mgs.datamodel.flight.FlightRawIndicatorValue;
import com.eurocopter.mgs.datamodel.health.common.Accelerometer;
import com.eurocopter.mgs.datamodel.health.common.ComponentAcquisition;
import com.eurocopter.mgs.datamodel.health.common.ContextualAcquisition;
import com.eurocopter.mgs.datamodel.health.common.HealthAlgorithm;
import com.eurocopter.mgs.datamodel.health.common.MarmsMechanicalComponentAcquisition;
import com.eurocopter.mgs.datamodel.health.common.SensorAcquisition;
import com.eurocopter.mgs.datamodel.health.common.Signal;
import com.eurocopter.mgs.datamodel.health.monitoring.HealthMechanicalComponent;
import com.eurocopter.mgs.datamodel.health.monitoring.HealthMechanicalComponentConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.HealthMonitoringConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.Indicator;
import com.eurocopter.mgs.datamodel.health.monitoring.IndicatorConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.TakeoffLandingDateTime;
import com.eurocopter.mgs.datamodel.util.PhysicalValue;
import com.eurocopter.mgs.download.analysis.exception.AnalysisException;
import com.eurocopter.mgs.download.analysis.health.api.IHealthHistoryProvider;
import com.eurocopter.mgs.download.analysis.health.data.IHealthAcquisition;
import com.eurocopter.mgs.download.analysis.health.data.IHealthIndicatorValueAcquisition;
import com.eurocopter.mgs.download.analysis.health.data.IHealthSession;
import com.eurocopter.mgs.download.analysis.health.dmau.util.HealthAcquisitionConstants;
import com.eurocopter.mgs.download.analysis.health.impl.filtering.AbstractFilteringRuleAlgorithm;
import com.eurocopter.mgs.download.analysis.health.impl.filtering.HelionixFilteringPerFileAlgorithm;
import com.eurocopter.mgs.download.analysis.health.impl.filtering.IFilteringRuleAlgorithm;
import com.eurocopter.mgs.download.analysis.health.impl.filtering.MarmsFilteringPerFlightHoursAlgorithm;
import com.eurocopter.mgs.download.analysis.health.util.HealthMonitoringAnalysisUtils;
import com.eurocopter.mgs.download.analysis.health.util.SignalEnvelopeCache;
import com.google.common.collect.ArrayListMultimap;
import com.google.common.collect.Multimap;

import corp.airbus.helicopters.algorithm.health.AverageSignalResult;
import corp.airbus.helicopters.algorithm.health.AverageSynchronousAlgorithm;
import corp.airbus.helicopters.algorithm.health.HealthAlgorithms;
import corp.airbus.helicopters.algorithm.health.IAverageSynchronousAlgorithm;
import corp.airbus.helicopters.algorithm.health.IHealthAlgorithms;
import corp.airbus.helicopters.algorithm.health.RejectedSignalException;
import corp.airbus.helicopters.algorithm.health.bearing.BearingAlgorithms;
import corp.airbus.helicopters.algorithm.health.bearing.BearingData;
import corp.airbus.helicopters.algorithm.health.bearing.BearingIndicator;
import corp.airbus.helicopters.algorithm.health.bearing.FnCommonProcessingResult;
import corp.airbus.helicopters.algorithm.health.bearing.IBearingAlgorithms;

public class HealthAcquisitionAnalyzer {

   private static final Logger LOGGER = LogManager.getLogger(HealthAcquisitionAnalyzer.class);

   /** The average synchronous algorithm */
   private static final IAverageSynchronousAlgorithm AVG_SYNC_ALGORITHM = new AverageSynchronousAlgorithm();

   /** The health algorithms */
   private static final IHealthAlgorithms HEALTH_ALGORITHMS = new HealthAlgorithms();

   /** The bearing algorithms */
   private static final IBearingAlgorithms BEARING_ALGORITHMS = new BearingAlgorithms();

   private static final String ENGINE = "ENGINE";

   void analyzeHealthAcquisitions(Aircraft aircraft, Flight flight, IHealthSession session, final HealthMonitoringConfiguration healthConfiguration,
         final Map<Indicator, FlightIndicatorValue> flightIndicatorValues, Map<Indicator, FlightRawIndicatorValue> flightIndicatorRawValues,
         final IHealthHistoryProvider healthHistoryProvider) throws AnalysisException {
      long startTime;
      Avionic avionic = aircraft.getAircraftType().getAvionic();

      // Map between source and active health mechanical components
      final Multimap<String, HealthMechanicalComponent> activeComponentMap = ArrayListMultimap.create();
      final Multimap<String, HealthMechanicalComponentConfiguration> activeComponentConfigurationMap = ArrayListMultimap.create();
      HealthMonitoringAnalysisUtils.indexComponentsBySource(avionic, healthConfiguration, activeComponentConfigurationMap, activeComponentMap);

      if (avionic == Avionic.CIGALHE) {
         // CIGALHE
         startTime = System.currentTimeMillis();
         analyzeSensorAcquisition(aircraft, flight, activeComponentConfigurationMap, flightIndicatorValues, flightIndicatorRawValues);
         LOGGER.info("Sensor acquisition analysis done in {} ms.", (System.currentTimeMillis() - startTime));
      } else {
         // M'ARMS
         startTime = System.currentTimeMillis();
         analyzeMarmsHealthAcquisitions(flight, session, healthConfiguration, activeComponentMap, flightIndicatorValues);
         LOGGER.info("MARMS health acquisition analysis done in {} ms.", (System.currentTimeMillis() - startTime));
      }
      startTime = System.currentTimeMillis();
      analyzeHealthIndicatorValueAcquisitions(avionic, flight, session, healthConfiguration, activeComponentMap, flightIndicatorValues, healthHistoryProvider);
      LOGGER.info("Indicator value aquisition analysis done in {} ms.", (System.currentTimeMillis() - startTime));
   }

   /**
    * Analyze sensor acquisition.
    *
    * @param aircraft
    *           the aircraft
    * @param flight
    *           the flight
    * @param componentConfigurationMap
    * @param indicatorValues
    *           the indicator values
    * @throws AnalysisException
    *           the analysis exception
    * @throws RejectedSignalException
    */
   private void analyzeSensorAcquisition(final Aircraft aircraft, final Flight flight, Multimap<String, HealthMechanicalComponentConfiguration> componentConfigurationMap,
         final Map<Indicator, FlightIndicatorValue> indicatorValues, final Map<Indicator, FlightRawIndicatorValue> flightRawIndicatorValues)
         throws AnalysisException {
      // Cached envelopes
      SignalEnvelopeCache envelopeCache = new SignalEnvelopeCache();

      Map<String, Map<Date, ContextualAcquisition>> contextualAcquisitions = HealthMonitoringAnalysisUtils.buildContextualAcquisitionsMap(flight);
      Map<ContextualAcquisition, Map<String, PhysicalValue>> maxCache = new HashMap<>();

      // Compute the average synchronous signal of the accelerometers
```

```
      for (final SensorAcquisition sensorAcquisition : flight.getSensorAcquisitions()) {
         final Date startDate = sensorAcquisition.getStartDate();

         String acquisitionName = sensorAcquisition.getAcquisitionName();
         // Retrieve active health mechanical component configurations relative to the acquisition name
         final Collection<HealthMechanicalComponentConfiguration> componentConfigurations = componentConfigurationMap.get(acquisitionName);

         // Update end date for ground acquisition display if necessary
         if (aircraft.getAircraftType().getAvionic() == Avionic.CIGALHE && (flight.getHealthEndDate() == null || flight.getHealthEndDate().before(startDate))) {
            flight.setHealthEndDate(startDate);
         }

         if (ENGINE.equalsIgnoreCase(sensorAcquisition.getAcquisitionType())) {
            continue;
         }

         // Index indicators by pairs <accelerometer/health mechanical component>
         final MultiValuedMap<AccelerometerMechanicalComponent, Indicator> indicatorMap =
HealthMonitoringAnalysisUtils.indexIndicatorsByAccelerometerAndHmc(componentConfigurations);

         final boolean groundAcquisition = flight.getFlightHourData() == null
               || !isAcquisitionPerformedDuringFlights(startDate, flight.getFlightHourData().getTakeoffLandingDateTimes());

         if (groundAcquisition && (flight.getHealthStartDate() == null || flight.getHealthStartDate().after(startDate))) {
            // PR 7856 : In this case, we will not be able to display the acquisition --> ignore acquisition
            continue;
         }

         final Map<String, PhysicalValue> contextualValues = HealthMonitoringAnalysisUtils.computeMaxContextualValues(contextualAcquisitions, maxCache, startDate,
acquisitionName);

         for (final Map.Entry<Accelerometer, ComponentAcquisition> entry : sensorAcquisition.getAcquisitions().entrySet()) {
            final Accelerometer accelerometer = entry.getKey();
            final ComponentAcquisition componentAcquisition = entry.getValue();

            // Compute the average synchronous signal
            for (final HealthMechanicalComponentConfiguration componentConfiguration : componentConfigurations) {
               final HealthMechanicalComponent component = componentConfiguration.getMechanicalComponent();

               // Because component cannot be modified (no cascade in healthComponentConf --> healthComponent relation)
               final HealthMechanicalComponent healthComponent = getHealthComponent(aircraft, component.getId());

               final AccelerometerMechanicalComponent key = new AccelerometerMechanicalComponent(accelerometer, component);
               final Collection<Indicator> indicators = indicatorMap.get(key);

               if (!indicators.isEmpty()) {
                  final double[] signal = componentAcquisition.getSignal().asDoubleArray();
                  final int[] topRoundMarkers = sensorAcquisition.getTopRoundMarkers().get(componentConfiguration.getPhaseSensor());

                  final AverageSignalResult averageSynchronousSignal;
                  try {
                     averageSynchronousSignal = AVG_SYNC_ALGORITHM.averageSynchronousSignal(signal, topRoundMarkers,
                           componentConfiguration.getInterpolationSize(), componentConfiguration.getRotationRatio(),
                           componentConfiguration.getConvergenceLimit());
                  } catch (final RejectedSignalException e) {
                     LOGGER.debug("Rejected signal [{}, {}]", accelerometer.getName(), component.getPath(), e);
                     continue;
                  } catch (final Exception e) {
                     LOGGER.debug("Error while computing average synchronous signal [{}, {}]", accelerometer.getName(), component.getPath(), e);
                     continue;
                  }

                  final double[] averageSignal = averageSynchronousSignal.getAverageSignal();
                  final double successRateValue = averageSynchronousSignal.getSuccessRate();

                  for (final Indicator indicator : indicators) {
                     if (componentConfiguration.getIndicatorConfigurations().containsKey(indicator)) {
                        IndicatorConfiguration indicatorConfiguration = componentConfiguration.getIndicatorConfigurations().get(indicator);
                        HealthAlgorithm algorithm = indicatorConfiguration.getHealthAlgorithm();

                        if (algorithm == HealthAlgorithm.Read) {
                           // It means a signal in the flight matches acquisition name of the mechanical component
                           // and accelerometer of the indicator but the current indicator is identified as "Read" => ignore the computation
                           continue;
                        }

                        if (algorithm == HealthAlgorithm.ACQUISITION_RATE) {
                           // It means a signal in the flight matches acquisition name of the mechanical component
                           // and accelerometer of the indicator but the current indicator is identified as "ACQUISITION_RATE"
                           // and computation must be carried out only once by flight hour => ignore the computation here
                           continue;
                        }

                        final List<Double> parameters;
                        if (algorithm == HealthAlgorithm.Fe || algorithm == HealthAlgorithm.Fi) {
                           parameters = new ArrayList<>();
                           // Compute and store "Nshaft" in parameter list
                           parameters.add(topRoundMarkers.length * componentConfiguration.getRotationRatio());
                           parameters.addAll(indicatorConfiguration.getParameters());
                        } else {
                           parameters = indicatorConfiguration.getParameters();
                        }

                        final double value;
                        try {
                           double samplingFrequency = sensorAcquisition.getSamplingFrequency();
                           double[] pulseTime = new double[signal.length];
                           for (int topRoundMarker : topRoundMarkers) {
                              pulseTime[topRoundMarker] = 1;
                           }
                           value = compute(algorithm, SignalType.PEAK, Avionic.CIGALHE, pulseTime, parameters, signal, averageSignal, envelopeCache, samplingFrequency);
                        } catch (RuntimeException | RejectedSignalException e) {
                           throw new AnalysisException("Could not compute value for indicator " + indicator.getFunctionalId(), e);
                        }

                        if (!Double.isNaN(value) && !Double.isInfinite(value)) {
                           if (indicatorConfiguration.getSlidingMedian() != null) {
                              HealthMonitoringAnalysisUtils.addIndicatorRawValue(flight, flightRawIndicatorValues, indicator, value, startDate);
                           } else {
                              HealthMonitoringAnalysisUtils.addIndicatorValue(flight, indicatorValues, indicator, value, startDate, successRateValue, null, contextualValues);
                           }
                        }
                     }
                  }
                  healthComponent.setGroundAcquisition(healthComponent.isGroundAcquisition() || groundAcquisition);
               }
            }
         }
      }
   }

   /**
    * Analyze marms health acquisitions.
    *
    * @param flight
    *           the flight
    * @param session
    *           the marms acq
    * @param healthConfiguration
    *           the health configuration
    * @param activeComponentMap
    * @param flightIndicatorValues
    *           the flight indicator values
    * @throws AnalysisException
    * @throws RejectedSignalException
    */
   private void analyzeMarmsHealthAcquisitions(final Flight flight, final IHealthSession session, final HealthMonitoringConfiguration healthConfiguration,
         Multimap<String, HealthMechanicalComponent> activeComponentMap, final Map<Indicator, FlightIndicatorValue> flightIndicatorValues)
         throws AnalysisException {
      // Cached envelopes
      final SignalEnvelopeCache envelopeCache = new SignalEnvelopeCache();
```

```
      for (final Entry<IHealthAcquisition, String> acqEntry : session.getHealthAcquisitions().entrySet()) {
         String source = acqEntry.getValue();
         for (final HealthMechanicalComponent component : activeComponentMap.get(source)) {
            IHealthAcquisition acq = acqEntry.getKey();
            addMechanicalComponentAcquisition(flight, component, source, acq);
            final HealthMechanicalComponentConfiguration hmcc = healthConfiguration.getMechanicalComponentConfigurations().get(component);
            for (final Entry<Indicator, IndicatorConfiguration> indicatorEntry : hmcc.getIndicatorConfigurations().entrySet()) {
               if (indicatorEntry.getValue().getHealthAlgorithm() == HealthAlgorithm.ACQUISITION_RATE) {
                  // The current indicator is identified as "ACQUISITION_RATE"
                  // and computation must be carried out only once by flight hour => ignore the computation here
                  continue;
               }
               computeIndicatorValues(flight, acq, indicatorEntry.getKey(), indicatorEntry.getValue(), flightIndicatorValues, envelopeCache);
            }
         }
      }
   }

   private void computeIndicatorValues(final Flight flight, IHealthAcquisition acq, final Indicator indicator, final IndicatorConfiguration indicatorConf,
         final Map<Indicator, FlightIndicatorValue> flightIndicatorValues, final SignalEnvelopeCache envelopeCache)
         throws AnalysisException {
      final HealthAlgorithm healthAlgorithm = indicatorConf.getHealthAlgorithm();
      for (Entry<String, Signal> signalEntry : acq.getAccelerometerSignals().entrySet()) {
         if (healthAlgorithm == HealthAlgorithm.Read) {
            // Ignore read indicators
            continue;
         }
         final double value;
         final double[] signal = signalEntry.getValue().asDoubleArray();
         String acquisitionTypeName = acq.getAcquisitionTypeName();
         try {
            double samplingFrequency = acq.getSamplingFrequency();
            if (HealthAcquisitionConstants.ACQ_MARMS_AVG.equals(acquisitionTypeName)) {
               value = compute(healthAlgorithm, SignalType.RMS, Avionic.MARMS, null, indicatorConf.getParameters(), null, signal, null, samplingFrequency);
            } else if (HealthAcquisitionConstants.ACQ_MARMS_RAW.equals(acquisitionTypeName)) {
               value = compute(healthAlgorithm, SignalType.RMS, Avionic.MARMS, null, indicatorConf.getParameters(), signal, null, envelopeCache, samplingFrequency);
            } else {
               throw new AnalysisException("Unknown M'ARMS aquisition type : " + acquisitionTypeName);
            }
         } catch (RuntimeException | RejectedSignalException e) {
            throw new AnalysisException("Could not compute value for indicator " + indicator.getFunctionalId(), e);
         }

         if (!Double.isNaN(value) && !Double.isInfinite(value)) {
            HealthMonitoringAnalysisUtils.addIndicatorValue(flight, flightIndicatorValues, indicator, value, acq.getStartDate(), null, acq.getCumulatedFlightTime(),
                  Collections.emptyMap());
         }
      }
   }

   private void analyzeHealthIndicatorValueAcquisitions(Avionic avionic, final Flight flight,
         final IHealthSession session, final HealthMonitoringConfiguration healthConfiguration,
         Multimap<String, HealthMechanicalComponent> activeComponentMap, final Map<Indicator, FlightIndicatorValue> flightIndicatorValues,
         final IHealthHistoryProvider healthHistoryProvider) {

      final IFilteringRuleAlgorithm filteringRuleAlgorithm;
      if (avionic == Avionic.CIGALHE) {
         filteringRuleAlgorithm = new HelionixFilteringPerFileAlgorithm();
      } else if (avionic == Avionic.MARMS) {
         filteringRuleAlgorithm = new MarmsFilteringPerFlightHoursAlgorithm(flight.getMaintenanceFlightTime());
      } else {
         throw new UnsupportedOperationException("Invalid avionic: " + avionic);
      }

      // BCR-6605 a deviation can override Points and the filtering law during acquisition filtering
      if (healthHistoryProvider != null && filteringRuleAlgorithm instanceof AbstractFilteringRuleAlgorithm<?> deviationAware) {
         deviationAware.setDeviations(healthHistoryProvider.getDeviations());
      }

      for (final IHealthIndicatorValueAcquisition valuesAcq : session.getHealthIndicatorValueAcquisitions()) {
         Date acqDate = valuesAcq.getStartDate();
         if (acqDate == null) {
            // Ignore acquisitions with invalid date (they cannot be displayed anyway)
            continue;
         }

         final Boolean groundAcquisition;
         if (avionic == Avionic.CIGALHE) {
            groundAcquisition = flight.getFlightHourData() == null || !isAcquisitionPerformedDuringFlights(acqDate, flight.getFlightHourData().getTakeoffLandingDateTimes());
            if (groundAcquisition && (flight.getHealthStartDate() == null || flight.getHealthStartDate().after(acqDate))) {
               // In this case, we will not be able to display the acquisition --> ignore acquisition
               continue;
            }
         } else {
            groundAcquisition = null;
         }

         final Collection<HealthMechanicalComponent> components;
         if (valuesAcq.isSourceRelevant()) {
            components = activeComponentMap.get(valuesAcq.getSource());
         } else {
            components = activeComponentMap.values();
         }
         for (final HealthMechanicalComponent component : components) {
            final HealthMechanicalComponentConfiguration hmcc = healthConfiguration.getMechanicalComponentConfigurations().get(component);
            if (hmcc.getFlightConfiguration() != null && hmcc.getFlightConfiguration().isEvm()) {
               continue;
            }

            // Update end date for ground acquisition display if necessary
            if (avionic == Avionic.CIGALHE && (flight.getHealthEndDate() == null || flight.getHealthEndDate().before(acqDate))) {
               flight.setHealthEndDate(acqDate);
            }

            boolean acquisitionFound = false;
            for (final Entry<Indicator, IndicatorConfiguration> entry : hmcc.getIndicatorConfigurations().entrySet()) {
               final Indicator indicator = entry.getKey();
               final IndicatorConfiguration conf = entry.getValue();
               final String indicatorId;
               if (conf.getHealthAlgorithm() == HealthAlgorithm.Read) {
                  indicatorId = conf.getReference();
               } else {
                  if (avionic == Avionic.CIGALHE) {
                     indicatorId = indicator.getFunctionalId();
                  } else {
                     final int parameter = conf.getParameters().isEmpty() ? 0 : conf.getParameters().get(0).intValue();
                     indicatorId = conf.getHealthAlgorithm().name() + parameter + conf.getAccelerometer();
                  }
               }
               if (!valuesAcq.getIndicatorValues().containsKey(indicatorId)) {
                  continue;
               }
               acquisitionFound = true;

               filteringRuleAlgorithm.recordAcquisition(valuesAcq, indicatorId, indicator, conf);
            }

            if (acquisitionFound && avionic == Avionic.CIGALHE) {
               hmcc.getMechanicalComponent().setGroundAcquisition(hmcc.getMechanicalComponent().isGroundAcquisition() || groundAcquisition);
            }
         }
      }

      for (HealthIndicatorValueAdapter healthIndicatorValueAdapter : filteringRuleAlgorithm.getFilteredValues(healthConfiguration)) {
         Double multiplier = healthIndicatorValueAdapter.getIndicatorConfiguration().getMultiplier();
         final double currentValue;
         if (multiplier != null) {
            currentValue = healthIndicatorValueAdapter.getValue() * multiplier;
         } else {
            currentValue = healthIndicatorValueAdapter.getValue();
         }
         HealthMonitoringAnalysisUtils.addIndicatorValue(flight, flightIndicatorValues, healthIndicatorValueAdapter.getIndicator(),
```

```
            currentValue, healthIndicatorValueAdapter.getAcquisitionDate(), healthIndicatorValueAdapter.getSuccessRate(),
            healthIndicatorValueAdapter.getCumulatedFlightTime(), healthIndicatorValueAdapter.getContextualValues());
      }
   }

   private HealthMechanicalComponent getHealthComponent(Aircraft aircraft, String id) {
      for (HealthMechanicalComponent healthMechanicalComponent : aircraft.getHealthComponents()) {
         if (healthMechanicalComponent.getId().equals(id)) {
            return healthMechanicalComponent;
         }
      }
      throw new NoSuchElementException("Health Mechanical Component not found: id=" + id);
   }

   /**
    * Return if an acquisition was performed during the flight phases.
    *
    * @param takeoffLandingDateTimes
    *           the list of take-off and landings
    * @return true if the acquisition was performed during the flight phases, false otherwise
    */
   private boolean isAcquisitionPerformedDuringFlights(final Date sensorAcqDateTime, final Set<TakeoffLandingDateTime> takeoffLandingDateTimes) {
      if (takeoffLandingDateTimes != null) {
         for (final TakeoffLandingDateTime takeoffLandingDateTime : takeoffLandingDateTimes) {
            if (sensorAcqDateTime.after(takeoffLandingDateTime.getTakeoffDateTime())
                  && sensorAcqDateTime.before(takeoffLandingDateTime.getLandingDateTime())) {
               return true;
            }
         }
      }
      return false;
   }

   /**
    * Compute.
    *
    * @param algorithm
    *           the algorithm
    * @param avionic
    * @param parameters
    *           the parameters
    * @param rawSignal
    *           the raw signal
    * @param averageSignal
    *           the average signal
    * @param envelopeCache
    *           the envelopes corresponding to the signal
    * @return the double
    * @throws AnalysisException
    * @throws RejectedSignalException
    */
   private double compute(final HealthAlgorithm algorithm, final SignalType signalType, Avionic avionic, double[] tops, final List<Double> parameters, final double[] rawSignal,
         final double[] averageSignal, final SignalEnvelopeCache envelopeCache, final double samplingFrequency)
         throws AnalysisException, RejectedSignalException {
      final double result;
      switch (algorithm) {
         case OM:
            if (averageSignal == null) {
               throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
            }
            result = HEALTH_ALGORITHMS.module(HEALTH_ALGORITHMS.fft(averageSignal), parameters.get(0).intValue()) / signalType.getCoefficient();
            break;
         case RMS:
            if (averageSignal == null) {
               throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
            }
            result = HEALTH_ALGORITHMS.rms(averageSignal);
            break;
         case RMSB:
            if (rawSignal == null) {
               throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
            }
            result = HEALTH_ALGORITHMS.rms(rawSignal);
            break;
         case Mod:
            if (averageSignal == null) {
               throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
            }
            result = HEALTH_ALGORITHMS.modulation(HEALTH_ALGORITHMS.fft(averageSignal), parameters.get(0).intValue()) / signalType.getCoefficient();
            break;
         case ModX:
            if (averageSignal == null) {
               throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
            }
            result = HEALTH_ALGORITHMS.modX(HEALTH_ALGORITHMS.fft(averageSignal), parameters.get(0).intValue(), parameters.get(1).intValue()) / signalType.getCoefficient();
            break;
         case NormX:
            if (averageSignal == null) {
               throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
            }
            result = HEALTH_ALGORITHMS.normX(HEALTH_ALGORITHMS.fft(averageSignal), parameters.get(0).intValue(), parameters.get(1).intValue());
            break;
         case Fi:
            result = computeBearingIndicator(algorithm, signalType, parameters, rawSignal, envelopeCache, true);
            break;
         case Fe:
            result = computeBearingIndicator(algorithm, signalType, parameters, rawSignal, envelopeCache, false);
            break;
         case RMSRoXXRoYY:
            // CF definition of RMSRoXXRoYY
            result = HEALTH_ALGORITHMS.rms(computeResidualSignal(algorithm, parameters, averageSignal));
            break;
         case Kr:
            result = HEALTH_ALGORITHMS.kurtosis(computeResidualSignal(algorithm, parameters, averageSignal));
            break;
         case Km:
            if (averageSignal == null) {
               throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
            }
            result = HEALTH_ALGORITHMS.kurtosis(averageSignal);
            break;
         case Kg:
            result = HEALTH_ALGORITHMS.kg(averageSignal, computeResidualSignal(algorithm, parameters, averageSignal));
            break;
         case MN:
            if (rawSignal == null) {
               throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
            }
            result = HEALTH_ALGORITHMS.moment(rawSignal, parameters.get(0).intValue());
            break;
         case OA:
            if (avionic == Avionic.CIGALHE) {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
               }
               result = BEARING_ALGORITHMS.overallAcceleration(rawSignal, samplingFrequency, parameters.get(0), parameters.get(1));
            } else {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
               }
               result = BEARING_ALGORITHMS.overallAcceleration(rawSignal, parameters.get(2), parameters.get(0), parameters.get(1));
            }
            break;
         case BPFO:
            if (avionic == Avionic.CIGALHE) {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
               }
               double[] pulses = HEALTH_ALGORITHMS.getPulses(tops, samplingFrequency);
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeHelionixFnCommonResult(rawSignal, pulses, samplingFrequency);
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.helionixFn(BearingIndicator.E, parameters.get(0), parameters.get(7), data, commonResult);
```

```
            } else {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
               }
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeMarmsFnCommonResult(rawSignal, parameters.get(0));
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.marmsFn(BearingIndicator.E, parameters.get(7), data, commonResult);
            }
            break;
         case BPFI:
            if (avionic == Avionic.CIGALHE) {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
               }
               double[] pulses = HEALTH_ALGORITHMS.getPulses(tops, samplingFrequency);
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeHelionixFnCommonResult(rawSignal, pulses, samplingFrequency);
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.helionixFn(BearingIndicator.I, parameters.get(0), parameters.get(7), data, commonResult);
            } else {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
               }
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeMarmsFnCommonResult(rawSignal, parameters.get(0));
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.marmsFn(BearingIndicator.I, parameters.get(7), data, commonResult);
            }
            break;
         case FTF:
            if (avionic == Avionic.CIGALHE) {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
               }
               double[] pulses = HEALTH_ALGORITHMS.getPulses(tops, samplingFrequency);
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeHelionixFnCommonResult(rawSignal, pulses, samplingFrequency);
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.helionixFn(BearingIndicator.C, parameters.get(0), parameters.get(7), data, commonResult);
            } else {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
               }
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeMarmsFnCommonResult(rawSignal, parameters.get(0));
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.marmsFn(BearingIndicator.C, parameters.get(7), data, commonResult);
            }
            break;
         case BSF:
            if (avionic == Avionic.CIGALHE) {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
               }
               double[] pulses = HEALTH_ALGORITHMS.getPulses(tops, samplingFrequency);
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeHelionixFnCommonResult(rawSignal, pulses, samplingFrequency);
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.helionixFn(BearingIndicator.B, parameters.get(0), parameters.get(7), data, commonResult);
            } else {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
               }
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeMarmsFnCommonResult(rawSignal, parameters.get(0));
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.marmsFn(BearingIndicator.B, parameters.get(7), data, commonResult);
            }
            break;
         case BPFO_ENV:
            if (avionic == Avionic.CIGALHE) {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
               }
               double[] pulses = HEALTH_ALGORITHMS.getPulses(tops, samplingFrequency);
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeHelionixFnEnvCommonResult(rawSignal, pulses, samplingFrequency, parameters.get(8),
parameters.get(9));
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.helionixFnEnv(BearingIndicator.E, parameters.get(0), parameters.get(7), data, commonResult);
            } else {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
               }
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeMarmsFnEnvCommonResult(rawSignal, parameters.get(0), parameters.get(8), parameters.get(9),
parameters.get(10),
                     parameters.get(11));
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.marmsFnEnv(BearingIndicator.E, parameters.get(7), data, commonResult);
            }
            break;
         case BPFI_ENV:
            if (avionic == Avionic.CIGALHE) {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
               }
               double[] pulses = HEALTH_ALGORITHMS.getPulses(tops, samplingFrequency);
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeHelionixFnEnvCommonResult(rawSignal, pulses, samplingFrequency, parameters.get(8),
parameters.get(9));
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.helionixFnEnv(BearingIndicator.I, parameters.get(0), parameters.get(7), data, commonResult);
            } else {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
               }
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeMarmsFnEnvCommonResult(rawSignal, parameters.get(0), parameters.get(8), parameters.get(9),
parameters.get(10),
                     parameters.get(11));
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.marmsFnEnv(BearingIndicator.I, parameters.get(7), data, commonResult);
            }
            break;
         case FTF_ENV:
            if (avionic == Avionic.CIGALHE) {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
               }
               double[] pulses = HEALTH_ALGORITHMS.getPulses(tops, samplingFrequency);
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeHelionixFnEnvCommonResult(rawSignal, pulses, samplingFrequency, parameters.get(8),
parameters.get(9));
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.helionixFnEnv(BearingIndicator.C, parameters.get(0), parameters.get(7), data, commonResult);
            } else {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
               }
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeMarmsFnEnvCommonResult(rawSignal, parameters.get(0), parameters.get(8), parameters.get(9),
parameters.get(10),
                     parameters.get(11));
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.marmsFnEnv(BearingIndicator.C, parameters.get(7), data, commonResult);
            }
            break;
         case BSF_ENV:
            if (avionic == Avionic.CIGALHE) {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
               }
               double[] pulses = HEALTH_ALGORITHMS.getPulses(tops, samplingFrequency);
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeHelionixFnEnvCommonResult(rawSignal, pulses, samplingFrequency, parameters.get(8),
parameters.get(9));
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.helionixFnEnv(BearingIndicator.B, parameters.get(0), parameters.get(7), data, commonResult);
            } else {
               if (rawSignal == null) {
                  throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
               }
               FnCommonProcessingResult commonResult = BEARING_ALGORITHMS.computeMarmsFnEnvCommonResult(rawSignal, parameters.get(0), parameters.get(8), parameters.get(9),
parameters.get(10),
```

```
                    parameters.get(11));
               BearingData data = buildBearingData(parameters);
               result = BEARING_ALGORITHMS.marmsFnEnv(BearingIndicator.B, parameters.get(7), data, commonResult);
            }
            break;
         case FMODEF:
            if (rawSignal == null) {
               throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
            }
            result = HEALTH_ALGORITHMS.fmodef(
                  rawSignal,
                  parameters.get(1),
                  parameters.get(2),
                  parameters.get(0).intValue(),
                  samplingFrequency);
            break;
         default:
            throw new AnalysisException("Invalid algorithm: " + algorithm);
      }
      return result;
   }

   private BearingData buildBearingData(final List<Double> parameters) {
      final double bearingDiameter = parameters.get(1);
      final double rollingElementDiameter = parameters.get(2);
      final int rollingElementNumber = parameters.get(3).intValue();
      final double contactAngle = parameters.get(4);
      final double internalRingRotationFrequency = parameters.get(5);
      final double externalRingRotationFrequency = parameters.get(6);
      return new BearingData(bearingDiameter, rollingElementDiameter,
            rollingElementNumber, contactAngle, internalRingRotationFrequency,
            externalRingRotationFrequency);
   }

   private void addMechanicalComponentAcquisition(final Flight flight, final HealthMechanicalComponent healthMechanicalComponent, String source, final IHealthAcquisition acq) {
      final MarmsMechanicalComponentAcquisition acquisition = new MarmsMechanicalComponentAcquisition(UUID.randomUUID().toString());
      acquisition.setMechanicalComponent(healthMechanicalComponent);
      acquisition.setAcquisitionName(source);
      acquisition.setStartDate(acq.getStartDate());
      // No accelerometer for M'ARMS acquisition (and a single signal)
      final Signal signal = acq.getAccelerometerSignals().get(IHealthAcquisition.NULL_ACCELEROMETER);
      signal.setFlightId(flight.getId());
      acquisition.setSamplingSize(signal.getValue().length / 8);
      acquisition.setSamplingFrequency(acq.getSamplingFrequency());
      acquisition.setSignal(signal);
      flight.getMechanicalComponentAcquisitions().add(acquisition);
   }

   private double[] computeResidualSignal(final HealthAlgorithm algorithm, final List<Double> parameters, final double[] averageSignal) throws AnalysisException {
      if (averageSignal == null) {
         throw new AnalysisException("Algorithm " + algorithm + " not applicable: no average signal");
      }
      double[][] rawFft = HEALTH_ALGORITHMS.rawFFT(averageSignal);
      int[] lines = new int[parameters.size()];
      for (int i = 0; i < parameters.size(); i++) {
         lines[i] = parameters.get(i).intValue();
      }
      final int size = rawFft[1].length;
      for (int i = size / 2; i < size; ++i) {
         rawFft[1][i] *= -1;
      }
      double[][] residual = HEALTH_ALGORITHMS.residualComplex(rawFft, lines);
      return HEALTH_ALGORITHMS.inverseFFTComplex(residual);
   }

   private double computeBearingIndicator(final HealthAlgorithm algorithm, final SignalType signalType, final List<Double> parameters,
         final double[] rawSignal, final SignalEnvelopeCache envelopeCache, final boolean internal) throws AnalysisException {
      if (rawSignal == null) {
         throw new AnalysisException("Algorithm " + algorithm + " not applicable: no raw signal");
      }
      final double rotations = parameters.get(0);
      final BearingData bearingData = buildBearingData(parameters);
      final double[] envelope = envelopeCache.get(rawSignal);
      if (internal) {
         return BEARING_ALGORITHMS.internalBearingIndicator(envelope, rotations, bearingData) * signalType.getCoefficient();
      } else {
         return BEARING_ALGORITHMS.externalBearingIndicator(envelope, rotations, bearingData) * signalType.getCoefficient();
      }
   }
}
```